

Is correlation equivalent to convolution
if we rotate its output?



rotate180(Correlation(I, K)) == Convolution(I, K) ????

Is correlation equivalent to convolution if we rotate its output?

Input Image



Kernel

$$\begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}$$

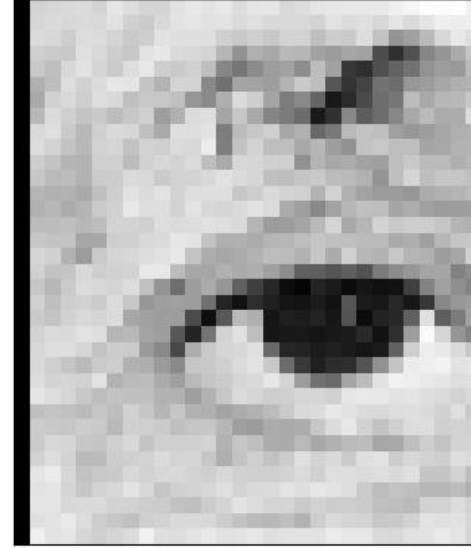
$Correlation(I, K)$

Einstein



$Convolution(I, K)$

Einstein



$rotate180(Correlation(I, K))$

Einstein Correlation - Rotated



$$rotate180(Correlation(I, K)) \neq Convolution(I, K)$$

Commutative property and padding in convolution



Let's review the example we saw the other day,
but using ***BORDER_REFLECT_101***...

Commutative property and padding in convolution

Convolution

Kernel H

1	2	3
4	5	6
7	8	9

★

Image F

0	0	0
0	1	0
0	0	0

BORDER_REFLECT_101

1	0	1	0	1
0	0	0	0	0
1	0	1	0	1
0	0	0	0	0
1	0	1	0	1

Commutative property and padding in convolution

Convolution

Kernel H

1	2	3
4	5	6
7	8	9

$\star$

Image F

0	0	0
0	1	0
0	0	0

$=$

9	8	7	
6	5	0	4
3	2	0	1
		0	0

$=$

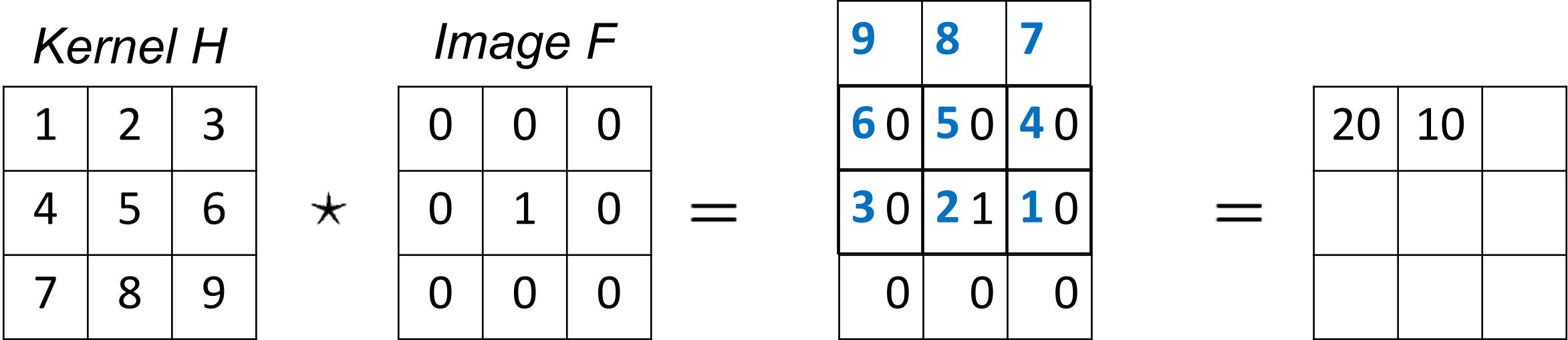
20		

BORDER_REFLECT_101

1	0	1	0	1
0	0	0	0	0
1	0	1	0	1
0	0	0	0	0
1	0	1	0	1

Commutative property and padding in convolution

Convolution

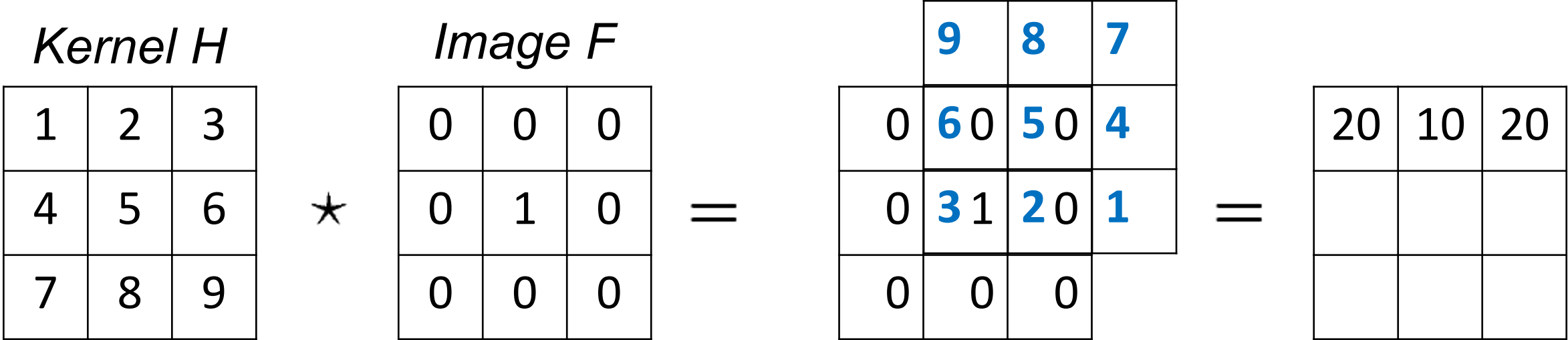


BORDER_REFLECT_101

1	0	1	0	1
0	0	0	0	0
1	0	1	0	1
0	0	0	0	0
1	0	1	0	1

Commutative property and padding in convolution

Convolution



BORDER_REFLECT_101

1	0	1	0	1
0	0	0	0	0
1	0	1	0	1
0	0	0	0	0
1	0	1	0	1

Commutative property and padding in convolution

Convolution

Kernel H

1	2	3
4	5	6
7	8	9

★

Image F

0	0	0
0	1	0
0	0	0

=

9	80	70	0
6	50	41	0
3	20	10	0

=

20	10	20
10		

BORDER_REFLECT_101

1	0	1	0	1
0	0	0	0	0
1	0	1	0	1
0	0	0	0	0
1	0	1	0	1

Commutative property and padding in convolution

Convolution

Kernel H

1	2	3
4	5	6
7	8	9

★

Image F

0	0	0
0	1	0
0	0	0

=

9 0	8 0	7 0
6 0	5 1	4 0
3 0	2 0	1 0

=

20	10	20
10	5	

BORDER_REFLECT_101

1	0	1	0	1
0	0	0	0	0
1	0	1	0	1
0	0	0	0	0
1	0	1	0	1

Commutative property and padding in convolution

Convolution

Kernel H

1	2	3
4	5	6
7	8	9

★

Image F

0	0	0
0	1	0
0	0	0

=

0	9	0	8	0	7
0	6	1	5	0	4
0	3	0	2	0	1

=

20	10	20
10	5	10

BORDER_REFLECT_101

1	0	1	0	1
0	0	0	0	0
1	0	1	0	1
0	0	0	0	0
1	0	1	0	1

Commutative property and padding in convolution

Convolution

Kernel H

1	2	3
4	5	6
7	8	9

Image F

0	0	0
0	1	0
0	0	0

★

=

		0	0	0
9	80	71	0	
6	50	40	0	
3	2	1		

=

20	10	20
10	5	10
20		

BORDER_REFLECT_101

1	0	1	0	1
0	0	0	0	0
1	0	1	0	1
0	0	0	0	0
1	0	1	0	1

Commutative property and padding in convolution

Convolution

Kernel H

1	2	3
4	5	6
7	8	9

Image F

0	0	0
0	1	0
0	0	0

★

=

0	0	0
9 0	8 1	7 0
6 0	5 0	4 0
3	2	1

=

20	10	20
10	5	10
20	10	

BORDER_REFLECT_101

1	0	1	0	1
0	0	0	0	0
1	0	1	0	1
0	0	0	0	0
1	0	1	0	1

Commutative property and padding in convolution

Convolution

Kernel H

1	2	3
4	5	6
7	8	9

★

Image F

0	0	0
0	1	0
0	0	0

=

0	0	0	
0	9 1	8 0	7
0	6 0	5 0	4
	3	2	1

=

20	10	20
10	5	10
20	10	20

BORDER_REFLECT_101

1	0	1	0	1
0	0	0	0	0
1	0	1	0	1
0	0	0	0	0
1	0	1	0	1

Commutative property and padding in convolution

Convolution

Kernel H

0	0	0
0	1	0
0	0	0

★

Image F

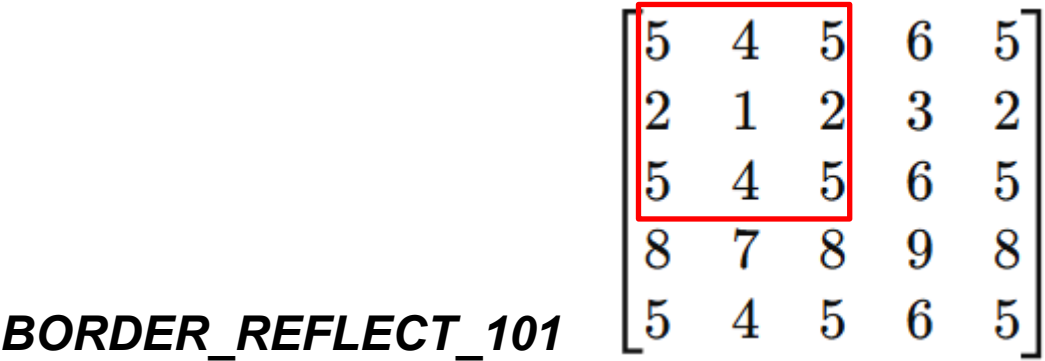
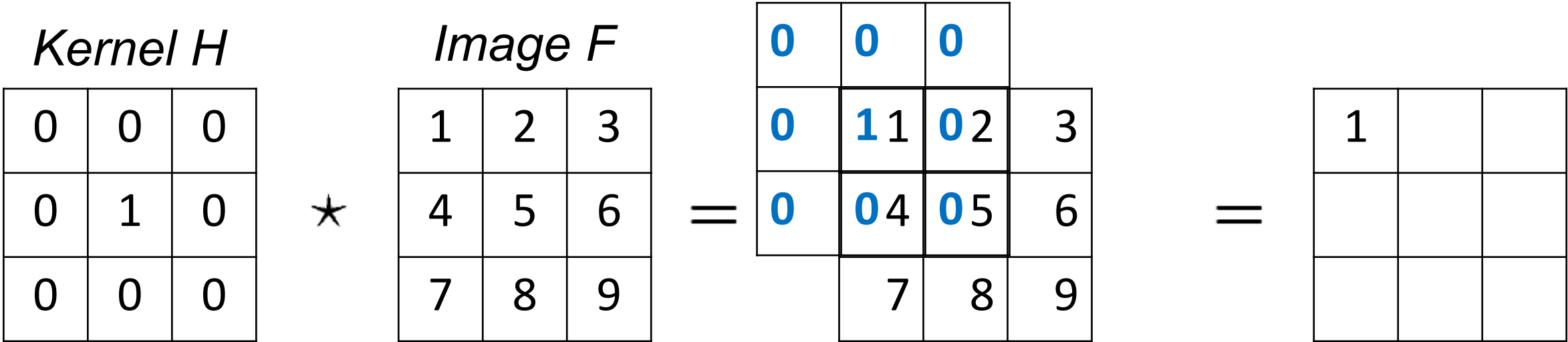
1	2	3
4	5	6
7	8	9

BORDER_REFLECT_101

5	4	5	6	5
2	1	2	3	2
5	4	5	6	5
8	7	8	9	8
5	4	5	6	5

Commutative property and padding in convolution

Convolution



Commutative property and padding in convolution

Convolution

Kernel H

0	0	0
0	1	0
0	0	0

$\star$

Image F

1	2	3
4	5	6
7	8	9

$=$

0	0	0
0 1	1 2	0 3
0 4	0 5	0 6
7	8	9

$=$

1	2	

BORDER_REFLECT_101

5	4	5	6	5
2	1	2	3	2
5	4	5	6	5
8	7	8	9	8
5	4	5	6	5

Commutative property and padding in convolution

Convolution

Kernel H

0	0	0
0	1	0
0	0	0

$\star$

Image F

1	2	3
4	5	6
7	8	9

$=$

	0	0	0
1	0 2	1 3	0
4	0 5	0 6	0
7	8	9	

$=$

1	2	3

BORDER_REFLECT_101

5	4	5	6	5
2	1	2	3	2
5	4	5	6	5
8	7	8	9	8
5	4	5	6	5

Commutative property and padding in convolution

Convolution

Kernel H

0	0	0
0	1	0
0	0	0

★

Image F

1	2	3
4	5	6
7	8	9

=

0	01	02	3
0	14	05	6
0	07	08	9

=

1	2	3
4		

BORDER_REFLECT_101

5	4	5	6	5
2	1	2	3	2
5	4	5	6	5
8	7	8	9	8
5	4	5	6	5

Commutative property and padding in convolution

Convolution

Kernel H

0	0	0
0	1	0
0	0	0

★

Image F

1	2	3
4	5	6
7	8	9

=

0 1	0 2	0 3
0 4	1 5	0 6
0 7	0 8	0 9

=

1	2	3
4	5	

BORDER_REFLECT_101

5	4	5	6	5
2	1	2	3	2
5	4	5	6	5
8	7	8	9	8
5	4	5	6	5

Commutative property and padding in convolution

Convolution

Kernel H

0	0	0
0	1	0
0	0	0

★

Image F

1	2	3
4	5	6
7	8	9

=

1	0	2	0	3	0
4	0	5	1	6	0
7	0	8	0	9	0

=

1	2	3
4	5	6

BORDER_REFLECT_101

5	4	5	6	5
2	1	2	3	2
5	4	5	6	5
8	7	8	9	8
5	4	5	6	5

Commutative property and padding in convolution

Convolution

Kernel H

0	0	0
0	1	0
0	0	0

★

Image F

1	2	3
4	5	6
7	8	9

=

	1	2	3
0	0 4	0 5	6
0	1 7	0 8	9
0	0	0	

=

1	2	3
4	5	6
7		

BORDER_REFLECT_101

5	4	5	6	5
2	1	2	3	2
5	4	5	6	5
8	7	8	9	8
5	4	5	6	5

Commutative property and padding in convolution

Convolution

Kernel H

0	0	0
0	1	0
0	0	0

★

Image F

1	2	3
4	5	6
7	8	9

=

1	2	3
0 4	0 5	0 6
0 7	1 8	0 9
0	0	0

=

1	2	3
4	5	6
7	8	

BORDER_REFLECT_101

5	4	5	6	5
2	1	2	3	2
5	4	5	6	5
8	7	8	9	8
5	4	5	6	5

Commutative property and padding in convolution

Convolution

Kernel H

0	0	0
0	1	0
0	0	0

★

Image F

1	2	3
4	5	6
7	8	9

=

1	2	3		
4	0	5	0	6
7	0	8	1	9
	0	0	0	

=

1	2	3
4	5	6
7	8	9

BORDER_REFLECT_101

5	4	5	6	5
2	1	2	3	2
5	4	5	6	5
8	7	8	9	8
5	4	5	6	5

Commutative property and padding in convolution

Convolution

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H & & \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} \star \begin{array}{|c|c|c|} \hline \textit{Image } F & & \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 20 & 10 & 20 \\ \hline 10 & 5 & 10 \\ \hline 20 & 10 & 20 \\ \hline \end{array}$$

$$\begin{array}{|c|c|c|} \hline \textit{Kernel } H & & \\ \hline 0 & 0 & 0 \\ \hline 0 & 1 & 0 \\ \hline 0 & 0 & 0 \\ \hline \end{array} \star \begin{array}{|c|c|c|} \hline \textit{Image } F & & \\ \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array} = \begin{array}{|c|c|c|} \hline 1 & 2 & 3 \\ \hline 4 & 5 & 6 \\ \hline 7 & 8 & 9 \\ \hline \end{array}$$

Non Commutative!!!!



It sounds counterintuitive, right?
Convolution is commutative, but it depends on the padding!!!

Commutative property and padding in convolution

Convolution

Kernel H Image F

0	0	0	0	0
0	1	2	3	0
0	4	5	6	0
0	7	8	9	0
0	0	0	0	0

★

1	0	1	0	1
0	0	0	0	0
1	0	1	0	1
0	0	0	0	0
1	0	1	0	1

=

20	10	20
10	5	10
20	10	20

Kernel H Image F

0	0	0	0	0
0	0	0	0	0
0	0	1	0	0
0	0	0	0	0
0	0	0	0	0

★

5	4	5	6	5
2	1	2	3	2
5	4	5	6	5
8	7	8	9	8
5	4	5	6	5

=

1	2	3
4	5	6
7	8	9

It sounds counterintuitive,
right?

**Convolution is commutative,
but it depends on the
padding!!!**

**The point is that, due to
padding, we are not
employing the exact
same signals!!!!**

We are just showing that

$$H * F \neq F' * H',$$
with $H \neq H'$ and $F \neq F'$

Commutative property and padding in convolution

Zero represents an absence of signal.

→ The convolution operation is mathematically defined over infinite signals.

For convolution to maintain some of its properties (commutativity, for instance), the surrounding space must be null.

If you use reflections or other padding strategies, you are introducing “synthetic information”.

→ Which can be useful at a practical level

→ But it deviates from the mathematical properties

What happens if we recursively apply *unsharp masking*?

Imagen Original

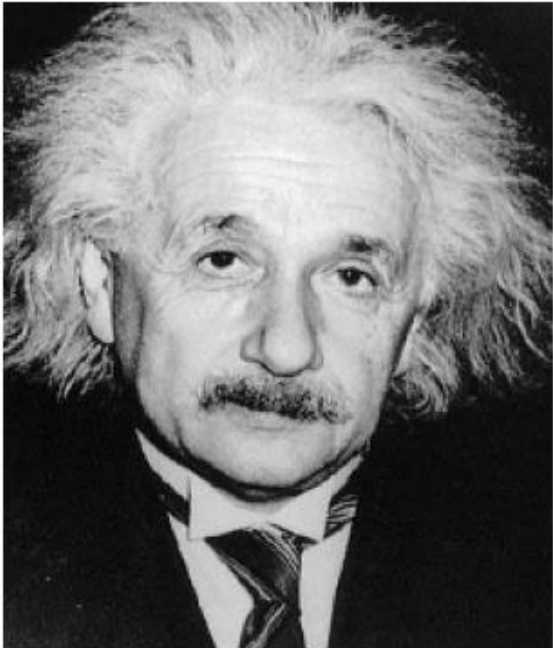
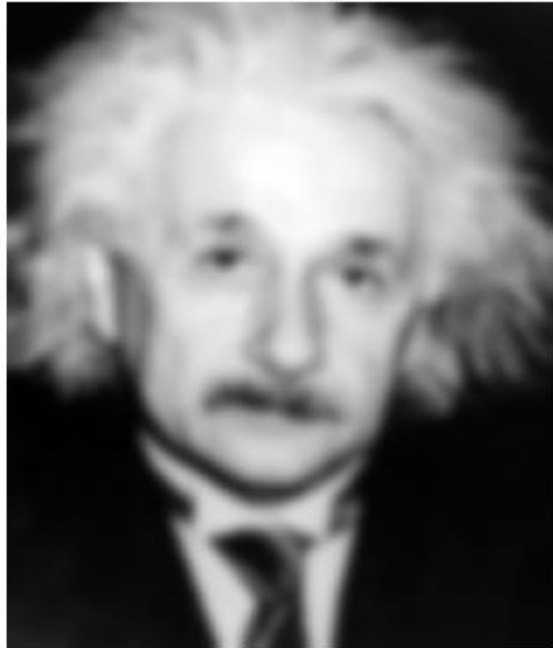
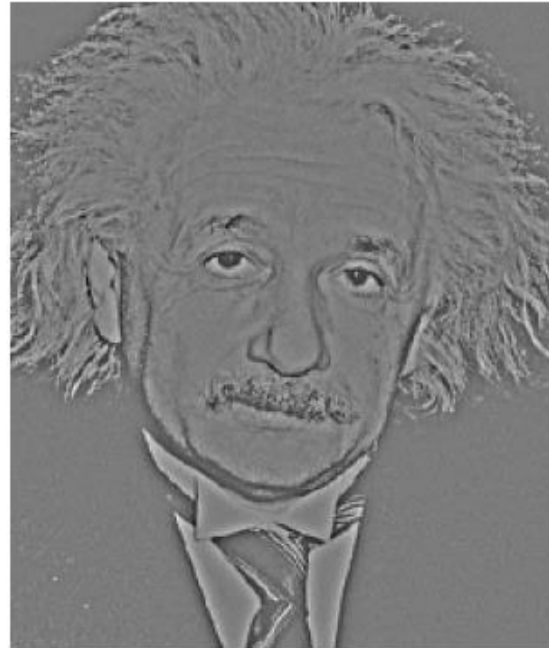


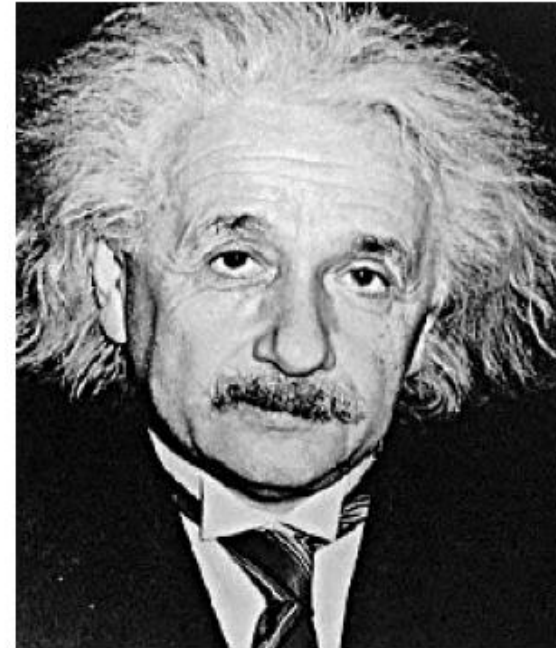
Imagen Suavizada (Bajas Frecuencias)



Original-Suavizada=AltasFrecuencias



Unsharp Masking



$$F + \alpha (F - F * H)$$

$\alpha = 1.0$

What happens if we recursively apply *unsharp masking*?

Imagen Original

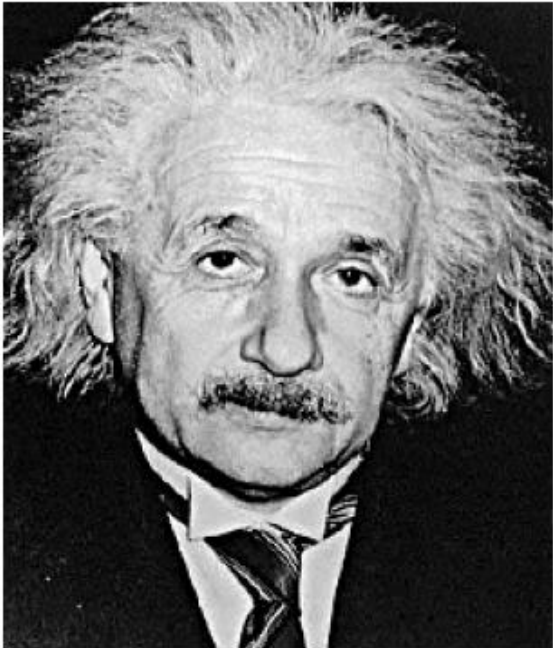
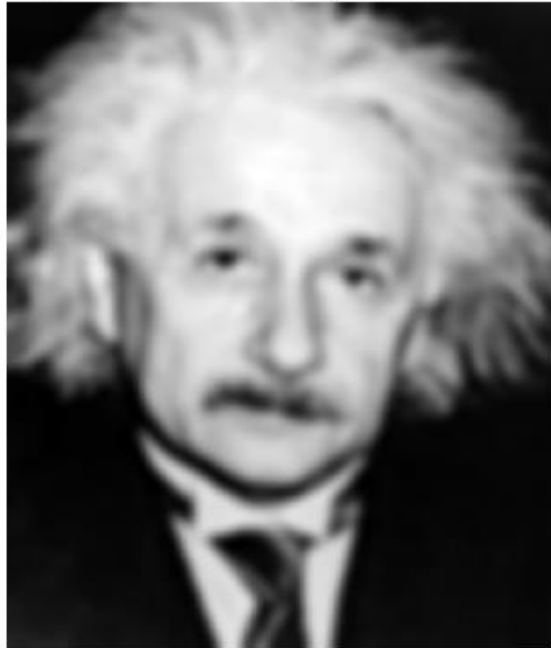
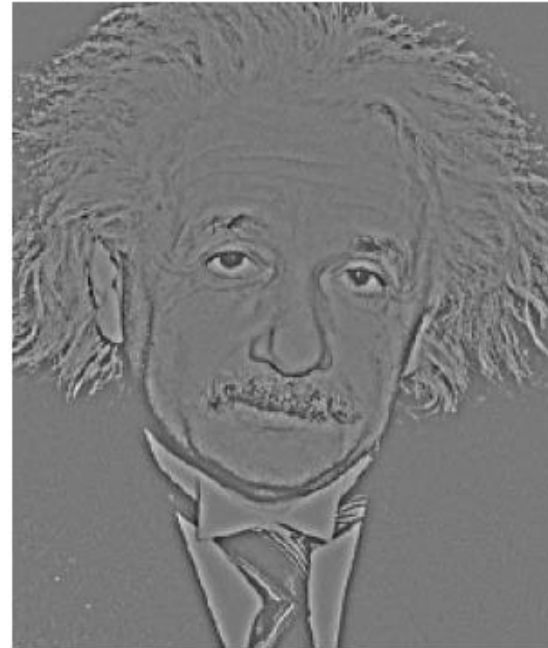


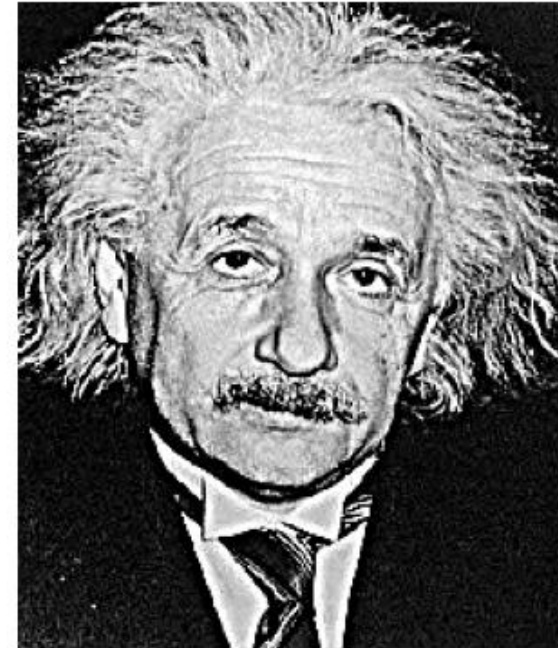
Imagen Suavizada (Bajas Frecuencias)



Original-Suavizada=AltasFrecuencias



Unsharp Masking



**As we discussed last time, the image eventually becomes saturated.
When the enhancement is too strong, it's as if we start introducing things that aren't there.**

What happens if we recursively apply *unsharp masking*?

Imagen Original

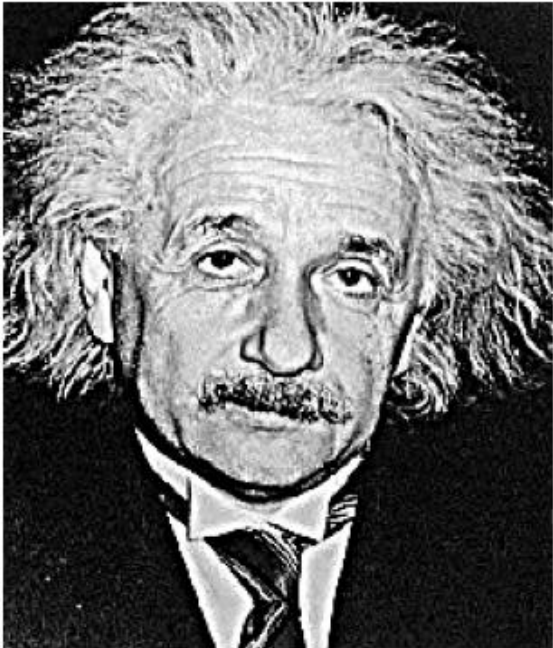
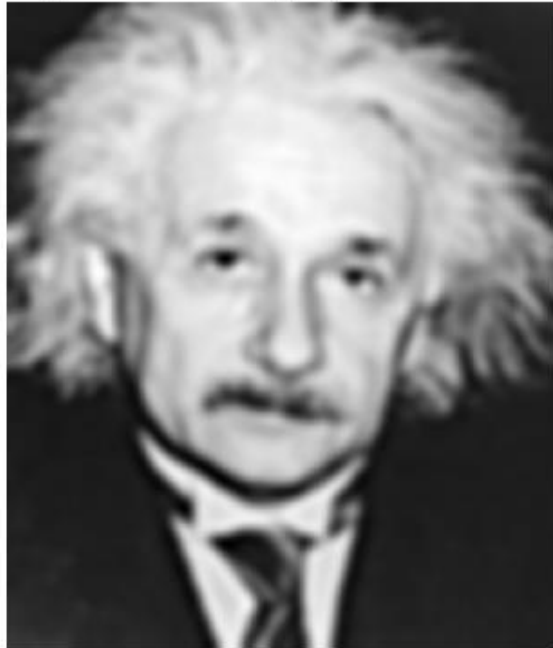
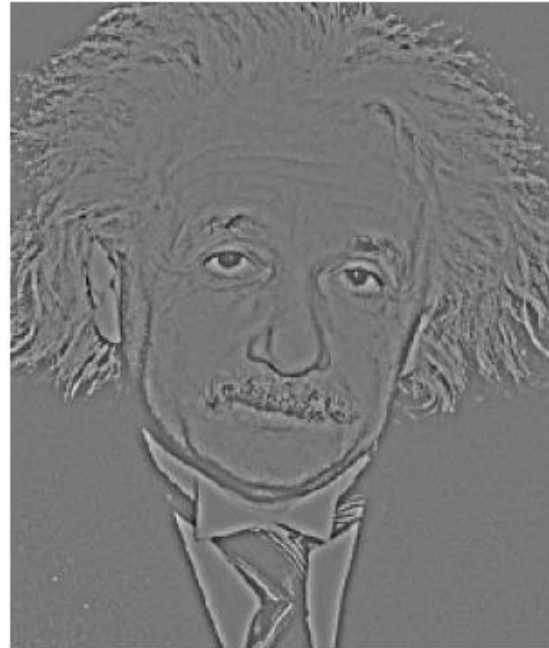


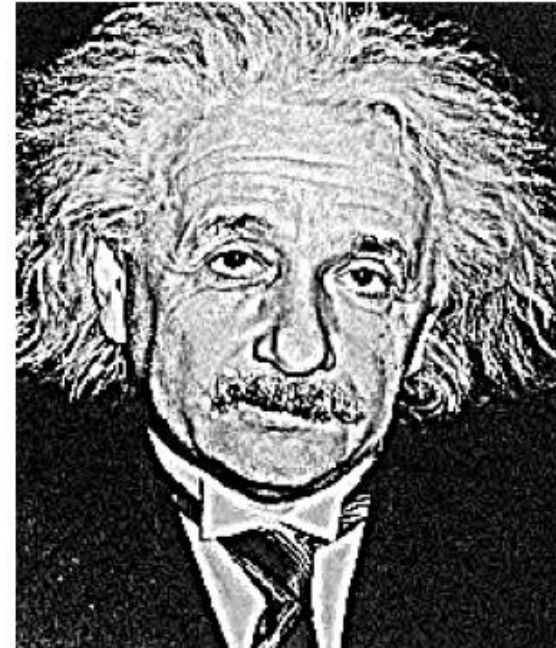
Imagen Suavizada (Bajas Frecuencias)



Original-Suavizada=AltasFrecuencias



Unsharp Masking



**As we discussed last time, the image eventually becomes saturated.
When the enhancement is too strong, it's as if we start introducing things that aren't there.**

What happens if we recursively apply *unsharp masking*?

Imagen Original

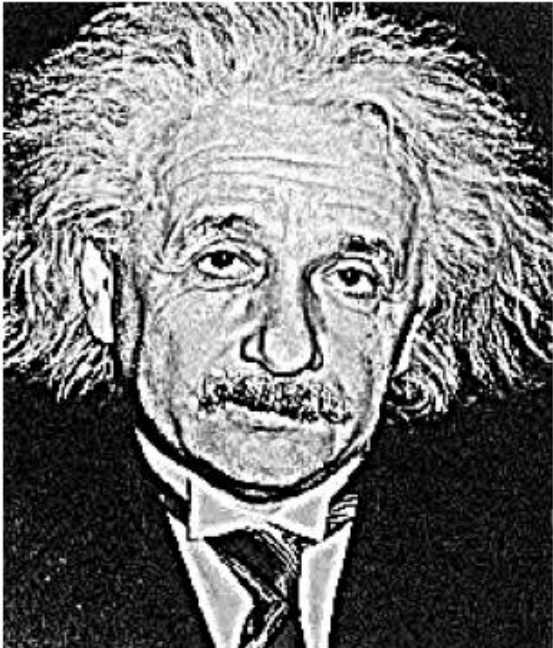
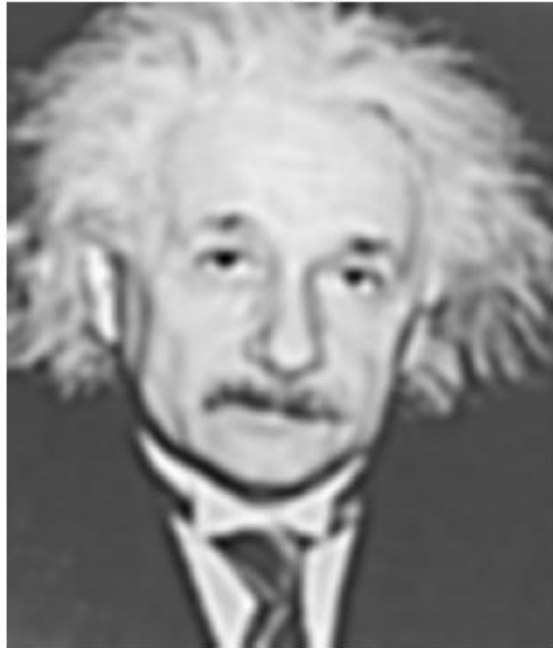
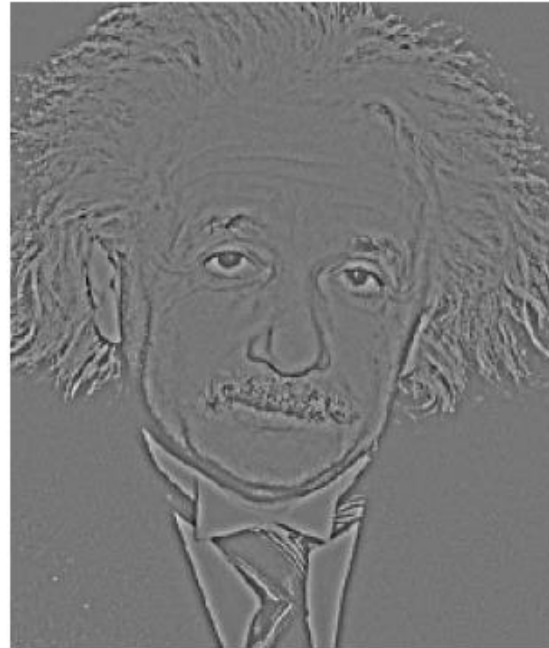


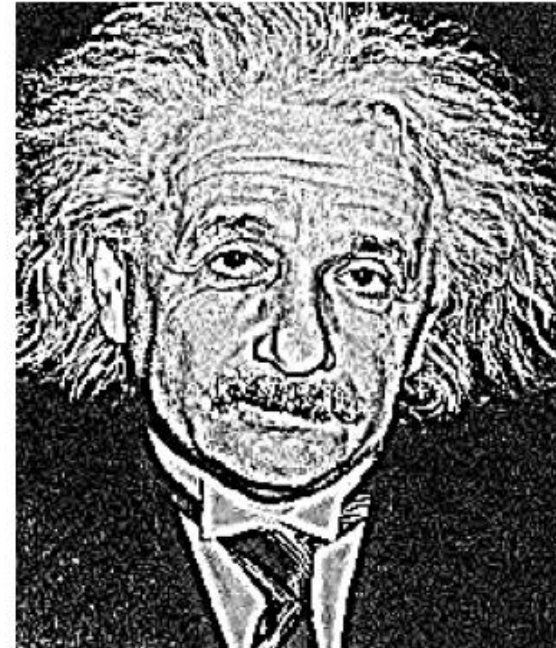
Imagen Suavizada (Bajas Frecuencias)



Original-Suavizada=AltasFrecuencias



Unsharp Masking



**As we discussed last time, the image eventually becomes saturated.
When the enhancement is too strong, it's as if we start introducing things that aren't there.**

What happens if we recursively apply *unsharp masking*?

Imagen Original

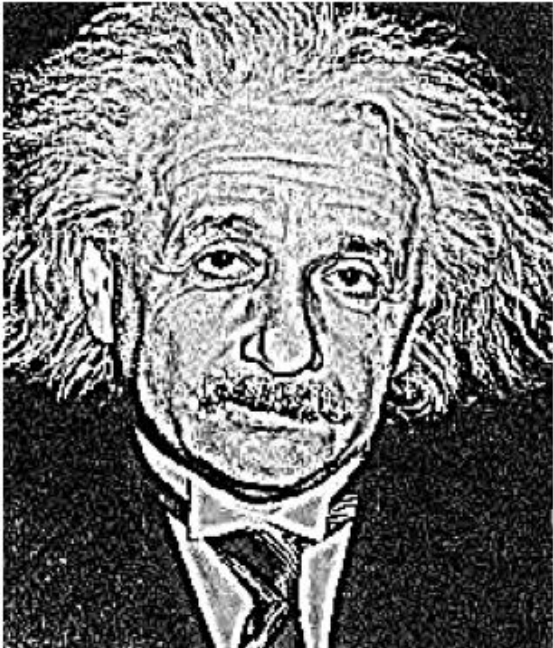
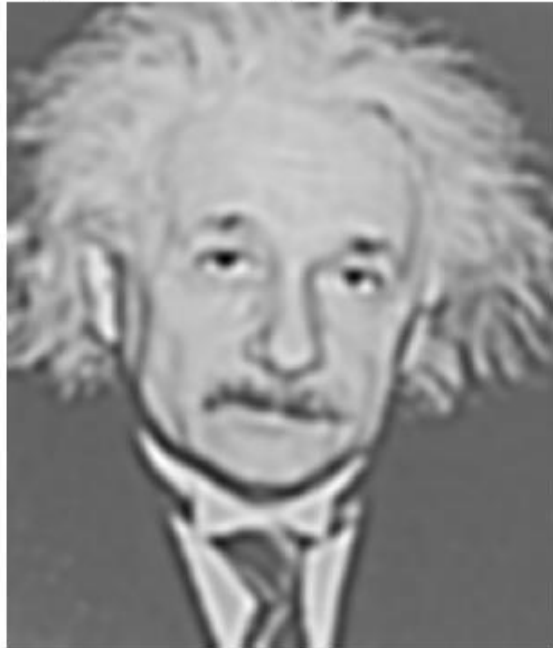
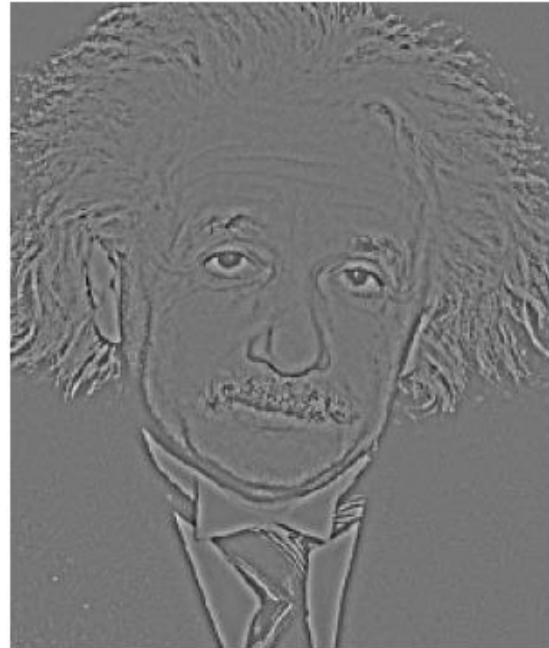


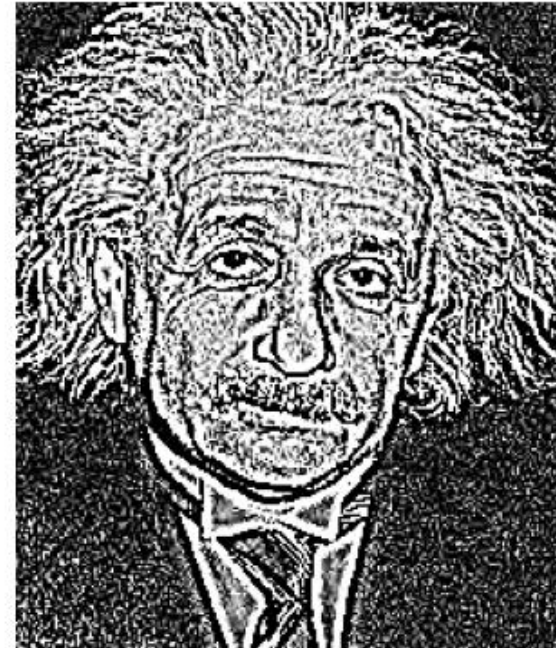
Imagen Suavizada (Bajas Frecuencias)



Original-Suavizada=AltasFrecuencias



Unsharp Masking



**As we discussed last time, the image eventually becomes saturated.
When the enhancement is too strong, it's as if we start introducing things that aren't there.**

What happens if we recursively apply *unsharp masking*?

Imagen Original

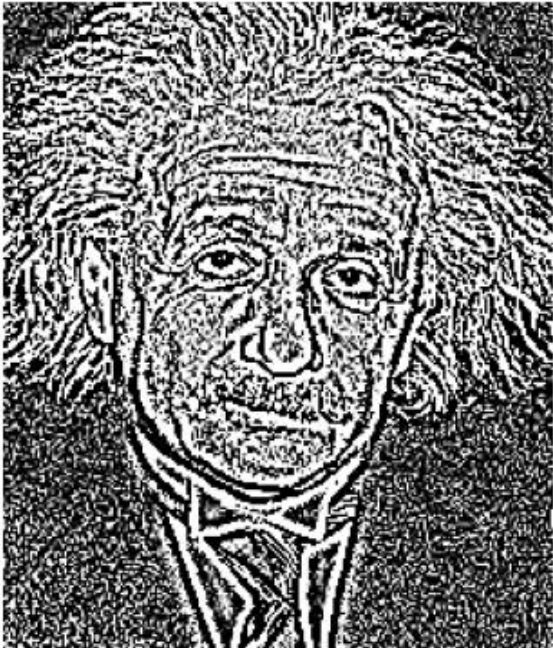
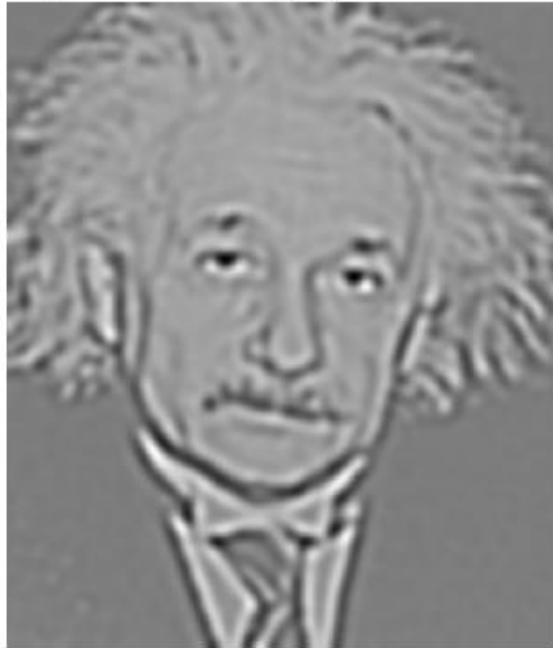
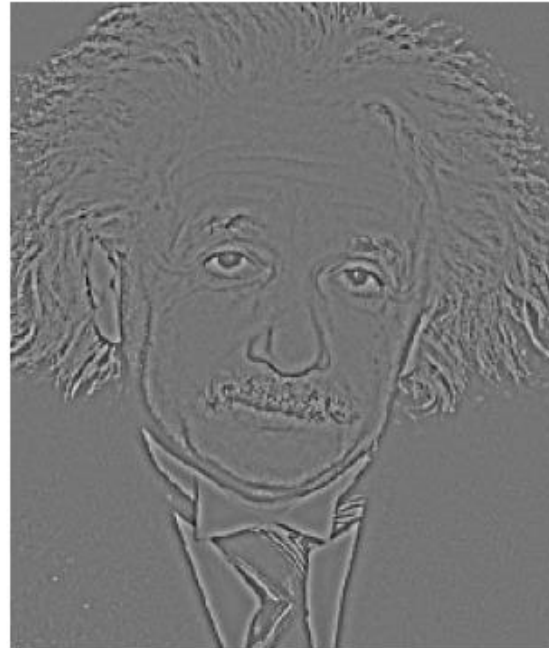


Imagen Suavizada (Bajas Frecuencias)



Original-Suavizada=AltasFrecuencias



Unsharp Masking



**As we discussed last time, the image eventually becomes saturated.
When the enhancement is too strong, it's as if we start introducing things that aren't there.**

What happens if we recursively apply *unsharp masking*?

Imagen Original

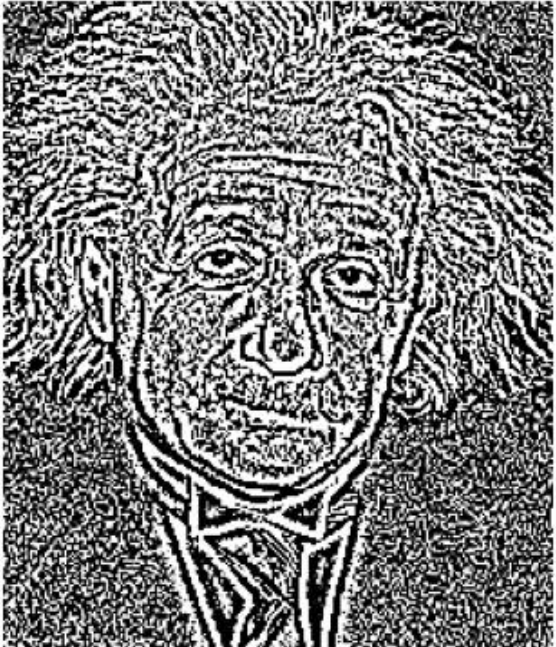
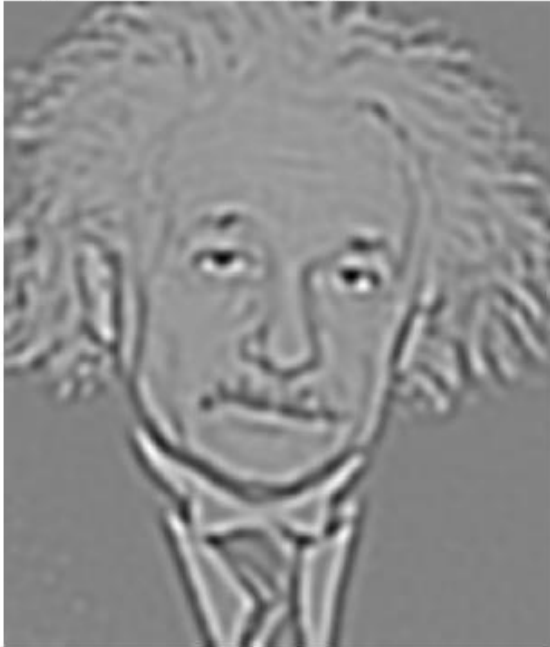
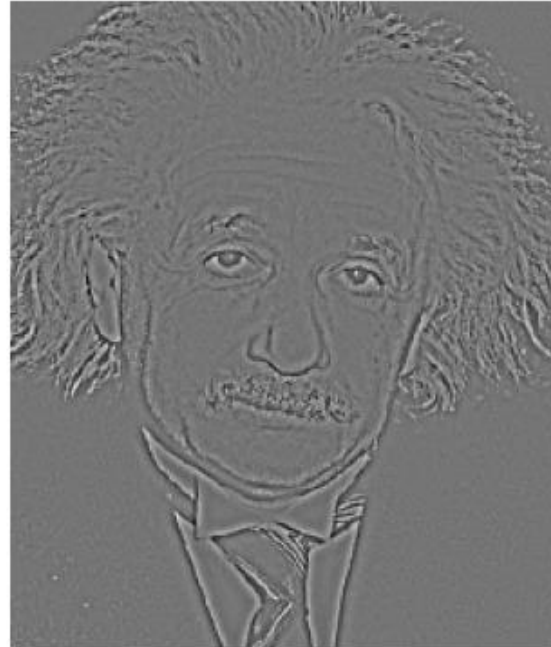


Imagen Suavizada (Bajas Frecuencias)



Original-Suavizada=AltasFrecuencias



Unsharp Masking



**As we discussed last time, the image eventually becomes saturated.
When the enhancement is too strong, it's as if we start introducing things that aren't there.**

What happens if we recursively apply *unsharp masking*?

100 runs later.....

As we discussed last time, the image eventually becomes saturated.
When the enhancement is too strong, it's as if we start introducing things that aren't there.

What happens if we recursively apply *unsharp masking*?

Imagen Original

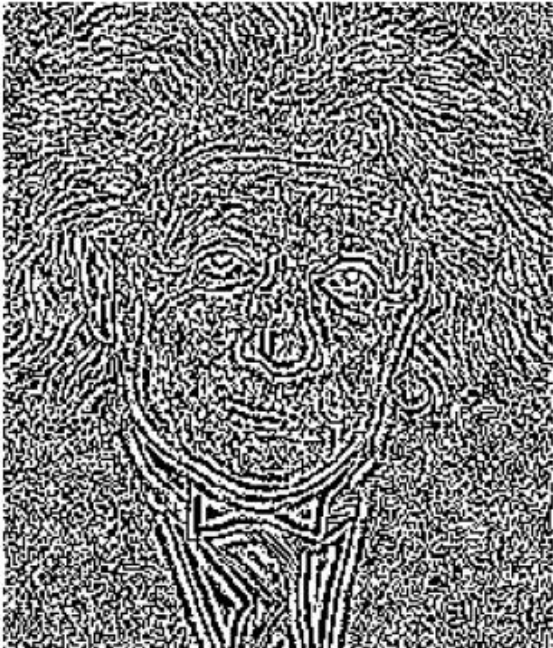


Imagen Suavizada (Bajas Frecuencias)



Original-Suavizada=AltasFrecuencias



Unsharp Masking



**As we discussed last time, the image eventually becomes saturated.
When the enhancement is too strong, it's as if we start introducing things that aren't there.**

Increasing α has a similar effect

Imagen Original

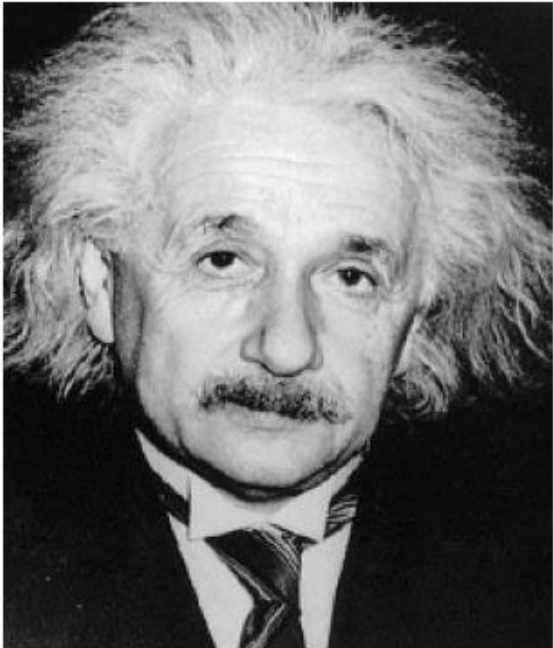
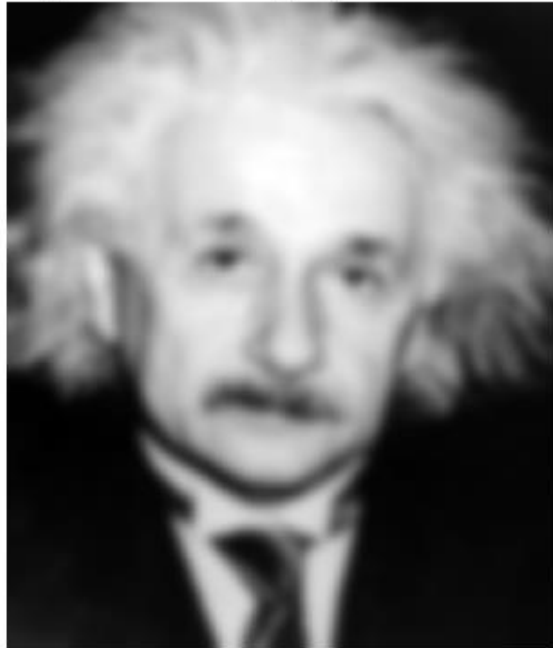
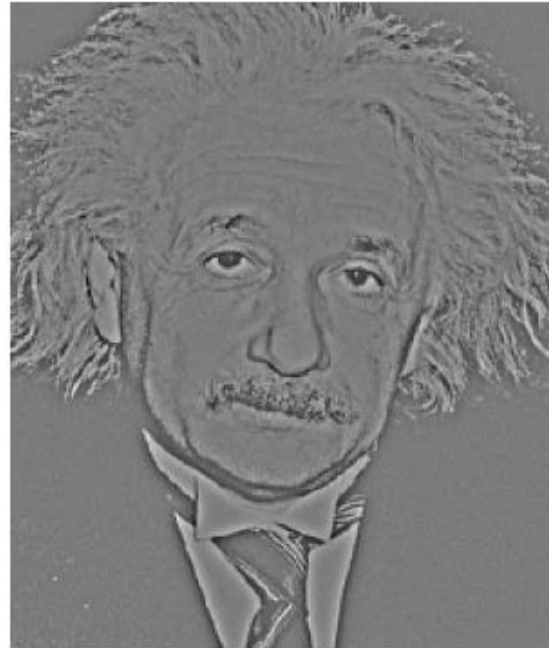


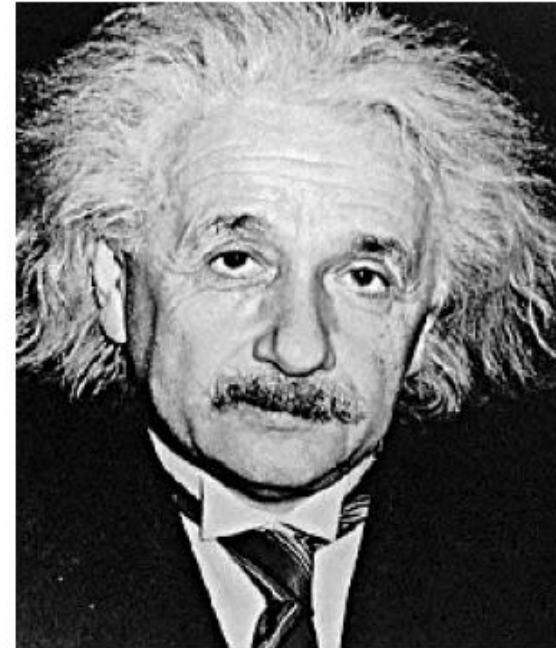
Imagen Suavizada (Bajas Frecuencias)



Original-Suavizada=AltasFrecuencias



Unsharp Masking



$$F + \alpha (F - F * H)$$

$\alpha = 1.0$

Increasing α has a similar effect

Imagen Original

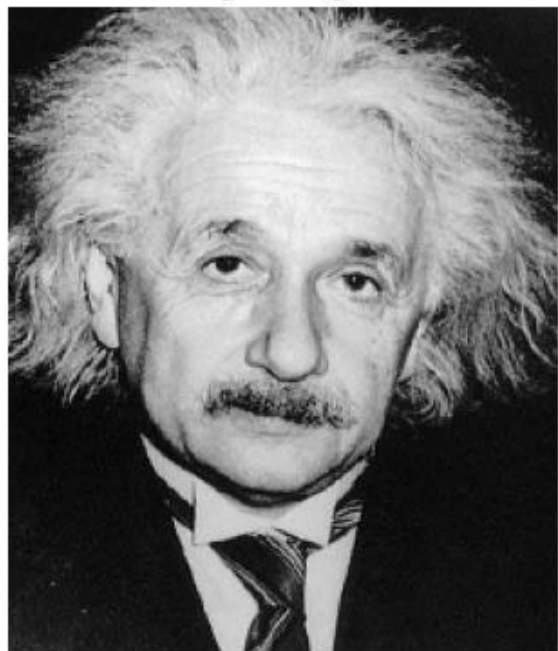
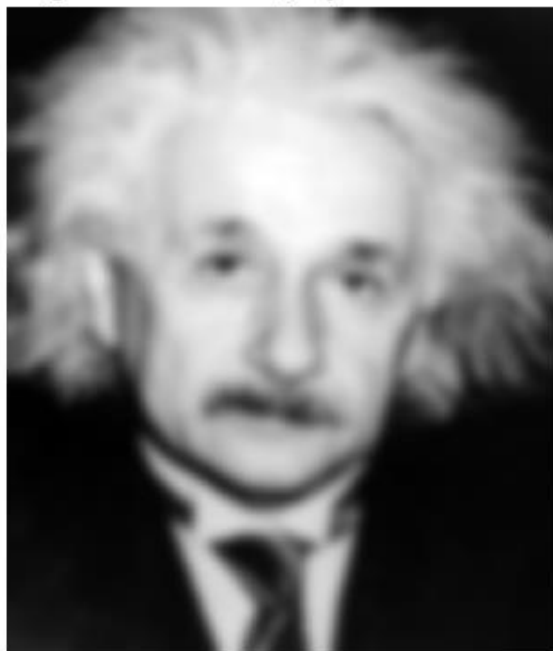
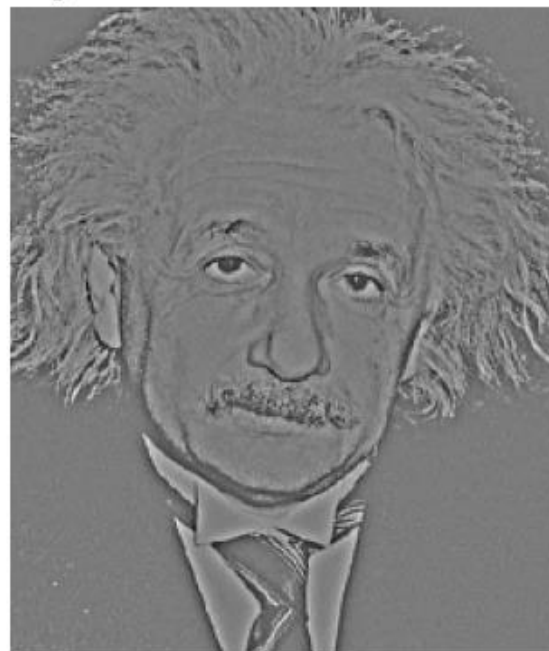


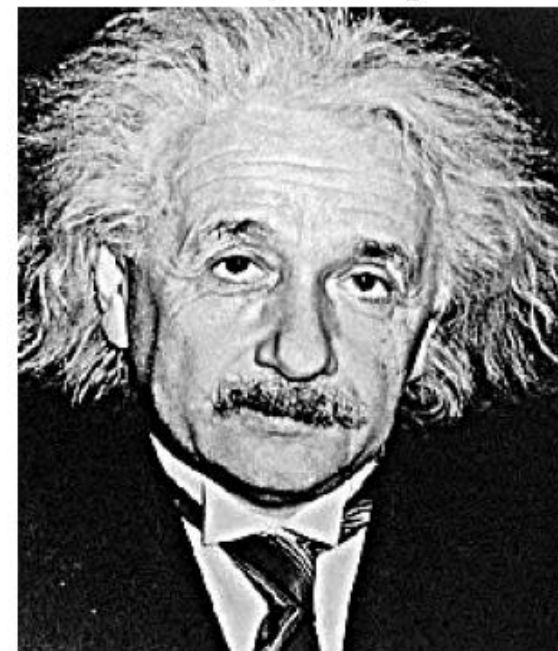
Imagen Suavizada (Bajas Frecuencias)



Original-Suavizada=AltasFrecuencias



Unsharp Masking



$$F + \alpha (F - F * H)$$

$\alpha = 2.0$

Increasing α has a similar effect

Imagen Original

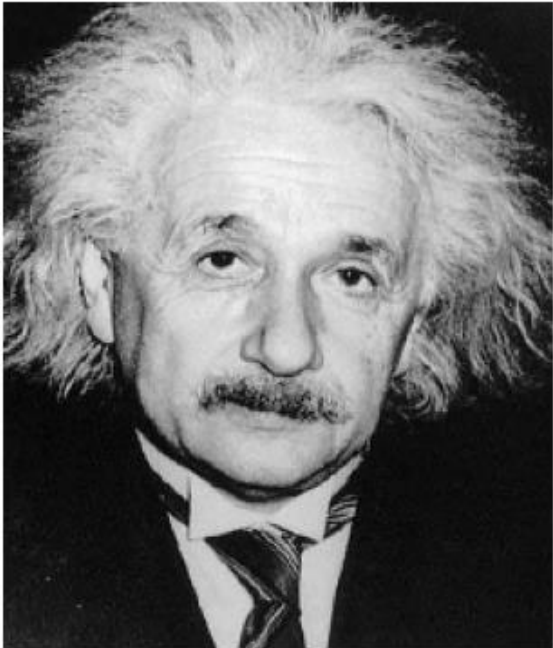
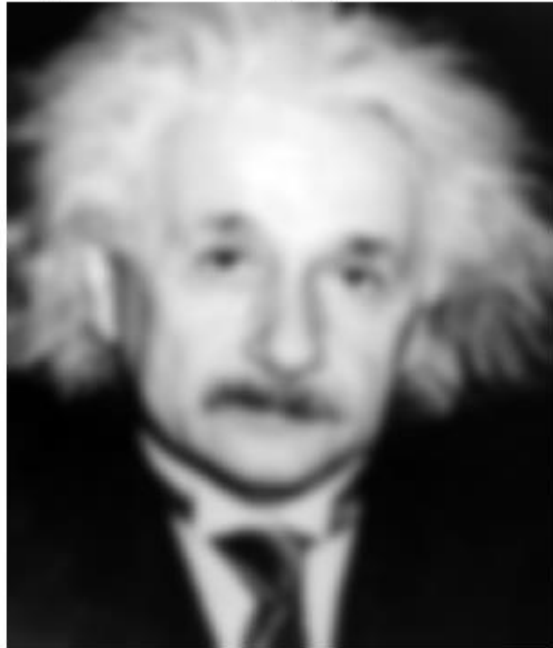
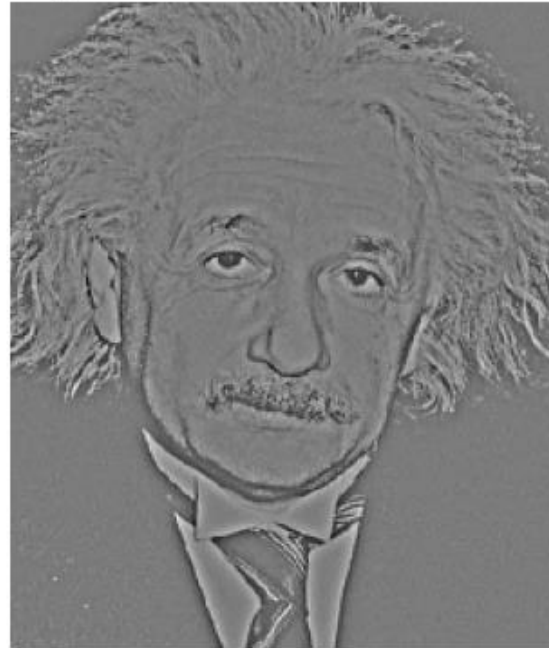


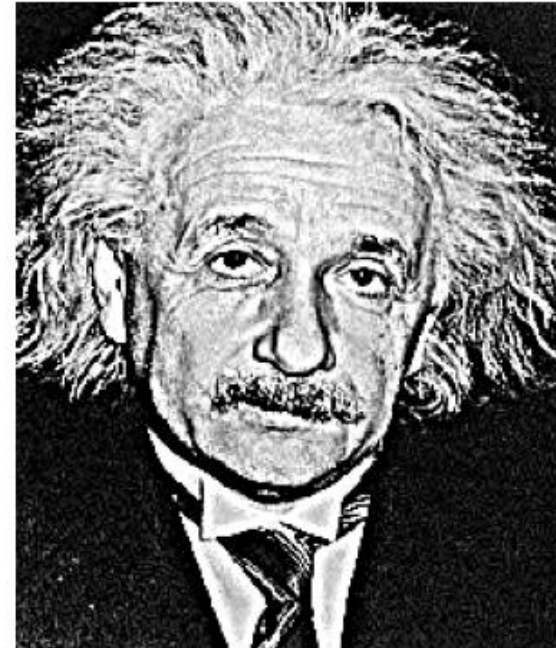
Imagen Suavizada (Bajas Frecuencias)



Original-Suavizada=AltasFrecuencias



Unsharp Masking



$$F + \alpha (F - F * H)$$

$\alpha = 5.0$

Increasing α has a similar effect

Imagen Original

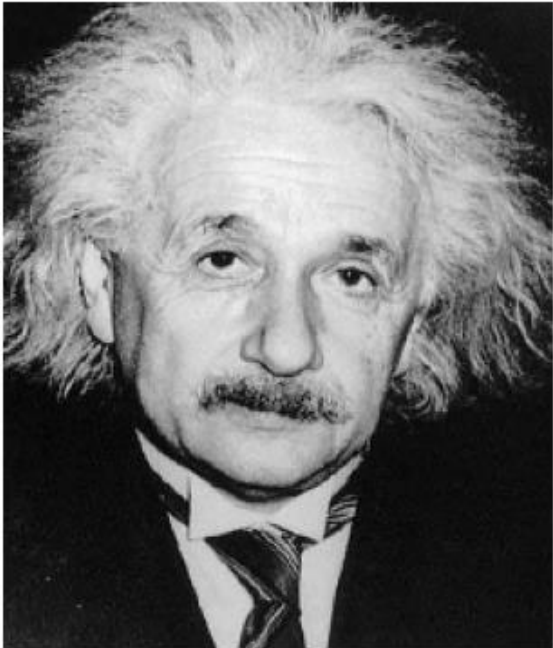
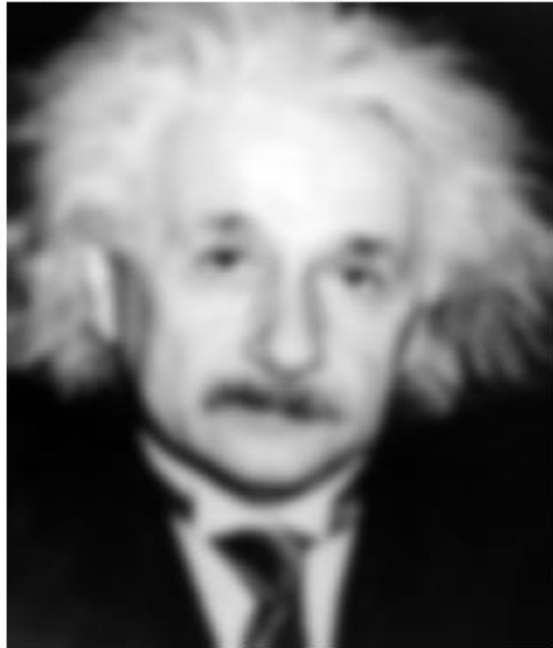
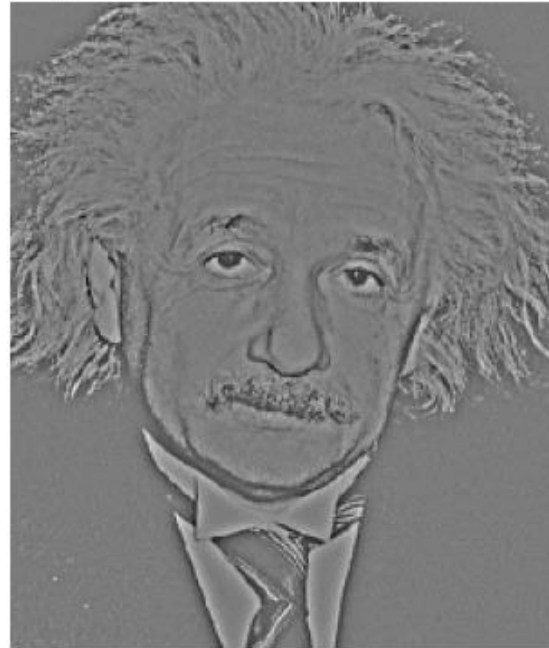


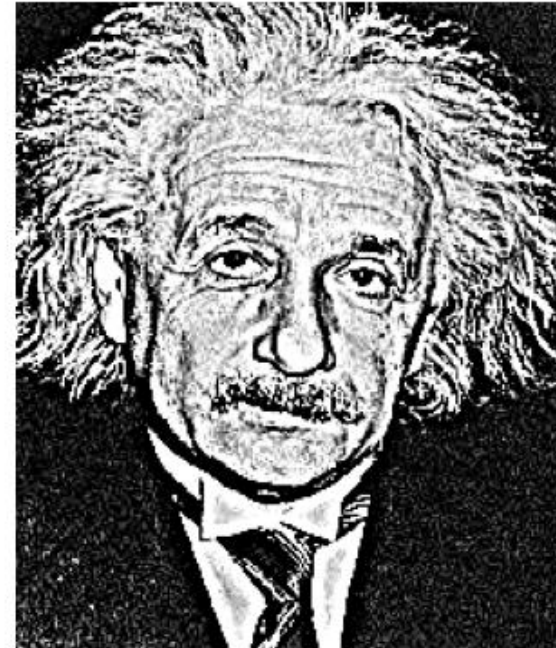
Imagen Suavizada (Bajas Frecuencias)



Original-Suavizada=AltasFrecuencias



Unsharp Masking



$$F + \alpha (F - F * H)$$

$\alpha = 10.0$

Increasing α has a similar effect

Imagen Original

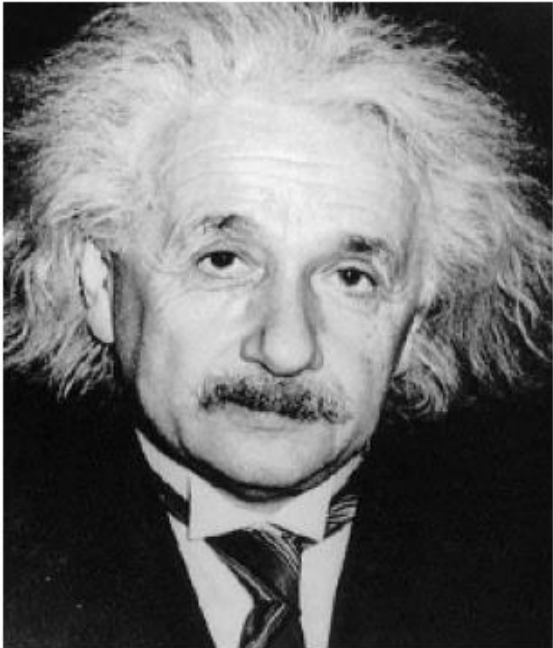
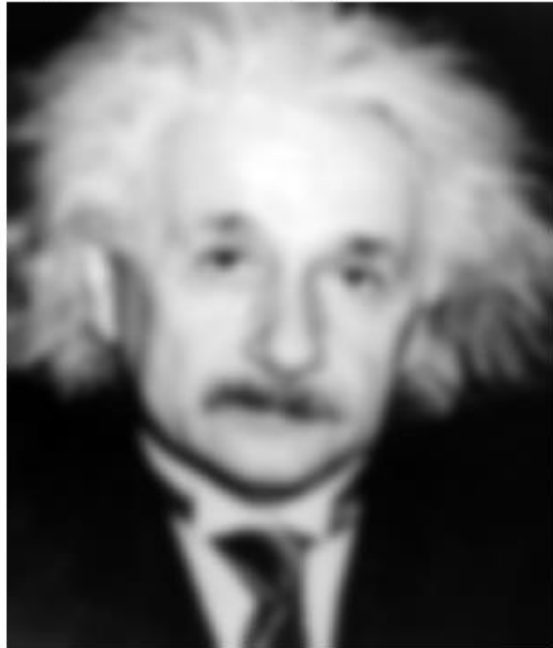
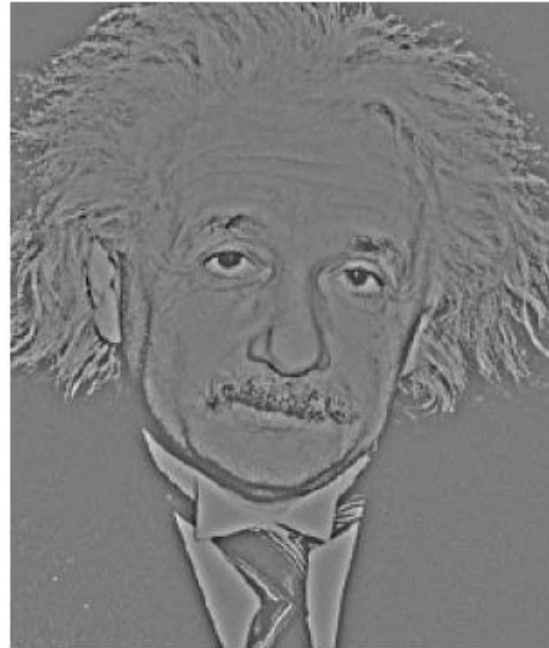


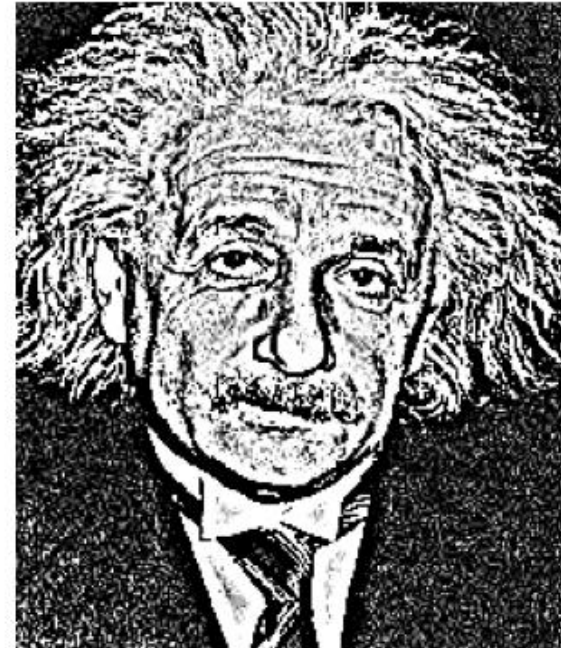
Imagen Suavizada (Bajas Frecuencias)



Original-Suavizada=AltasFrecuencias



Unsharp Masking



$$F + \alpha (F - F * H)$$

$\alpha = 25.0$

Increasing α has a similar effect

Imagen Original

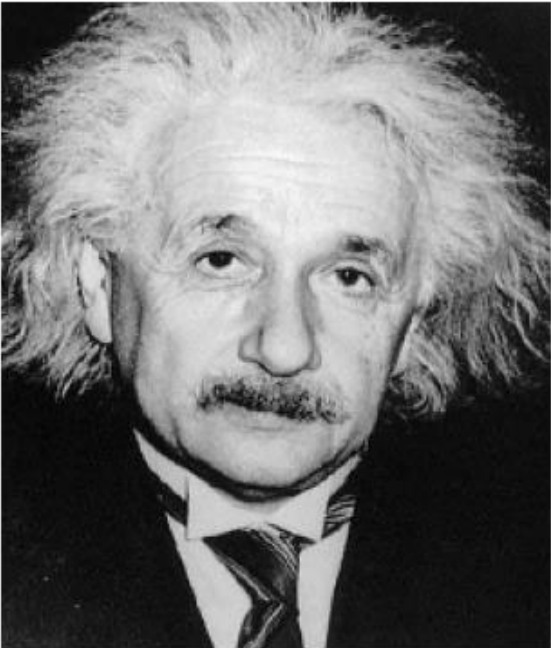
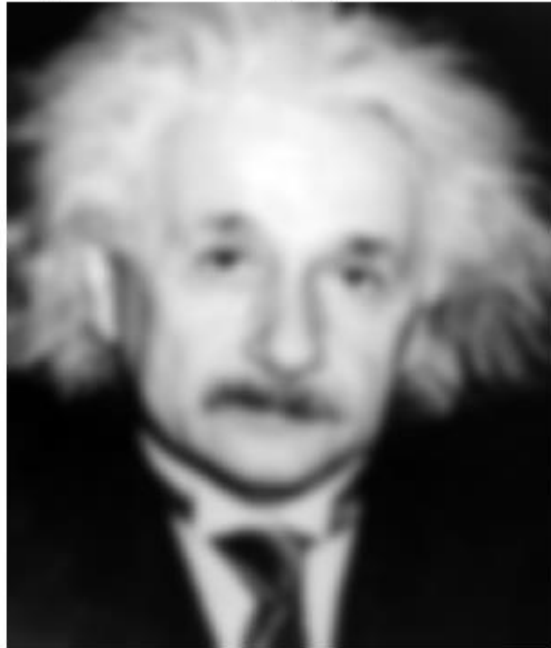
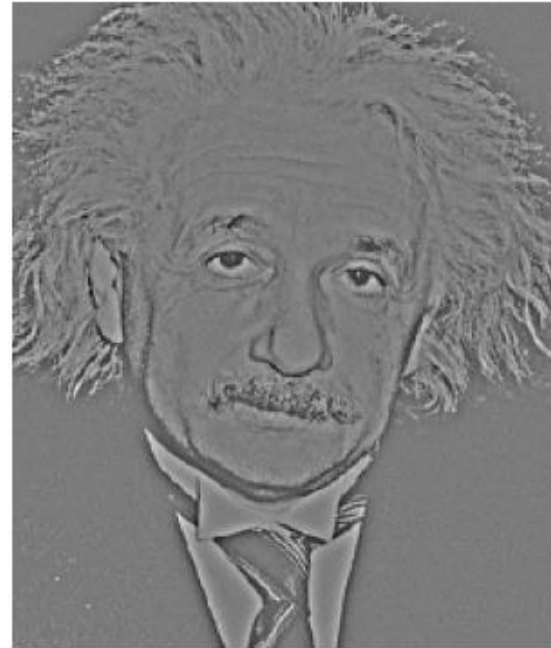


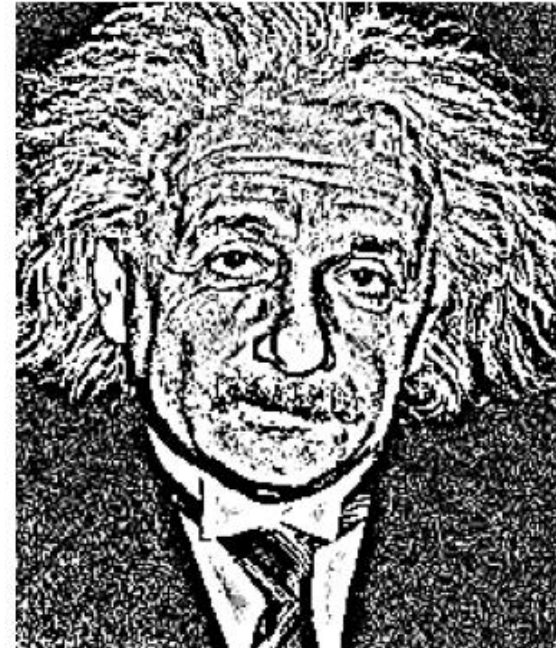
Imagen Suavizada (Bajas Frecuencias)



Original-Suavizada=AltasFrecuencias



Unsharp Masking



$$F + \alpha (F - F * H)$$

$\alpha = 50.0$

Increasing α has a similar effect

Imagen Original

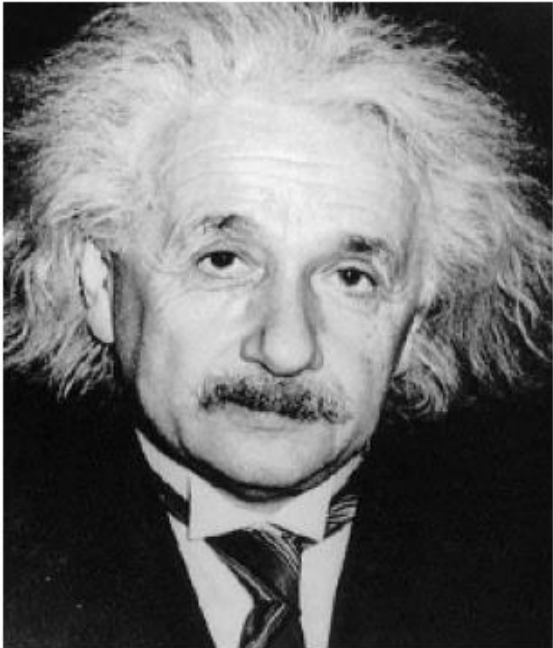
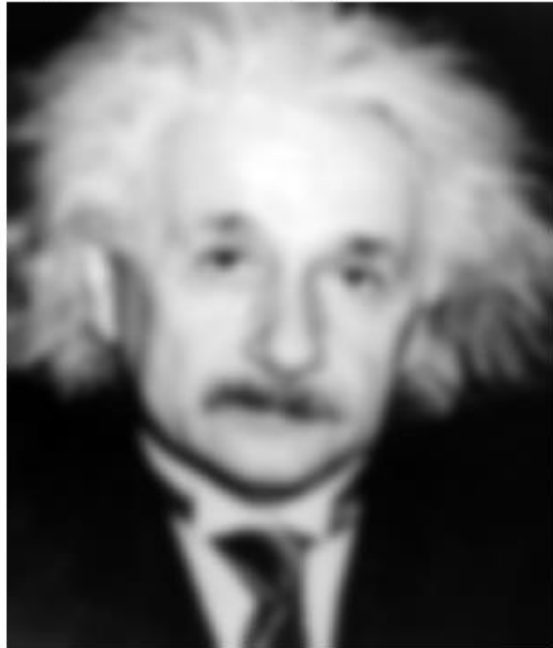
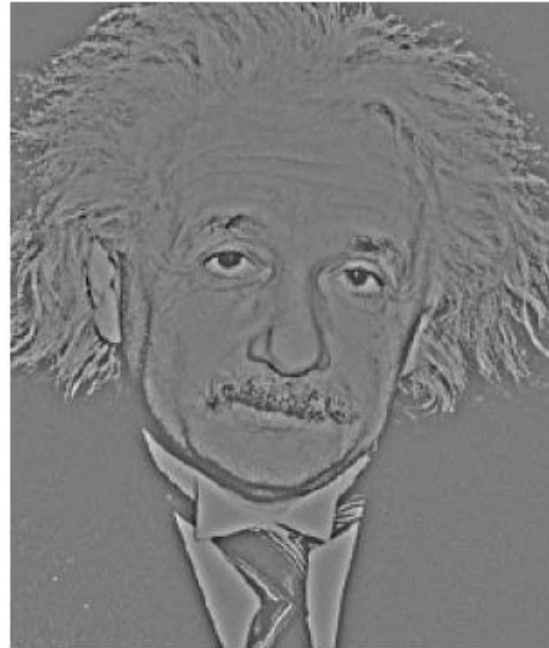


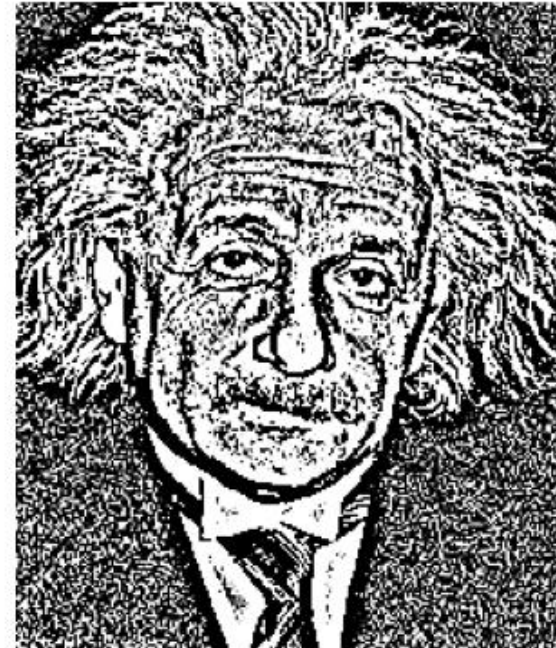
Imagen Suavizada (Bajas Frecuencias)



Original-Suavizada=AltasFrecuencias



Unsharp Masking



$$F + \alpha (F - F * H)$$

$\alpha = 99.0$

Is convolution equivariant to rotation and scaling?

First, what does “equivariance” mean?

`convolution(rotation(Img)) = rotation(convolution(Img))`

`convolution(scaling(Img)) = scaling(convolution(Img))`

Is convolution equivariant to translation?

Original Image



Shifted Image



1^o Convolution 2^o Shift



1^o Shift 2^o Convolution



$$\begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}$$

Is convolution equivariant to rotation?

Original Image



Rotated Image (90°)



1^o Convolution 2^o Rotation



1^o Rotation 2^o Convolution



$$\begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}$$

Is convolution equivariant to scaling?

Original Image (15, 15)



Scaled Image (300, 300)



1^o Convolution 2^o Scaling (300, 300)



1^o Scaling 2^o Convolution (300, 300)



$$\begin{pmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{pmatrix}$$

Linear filters

- Can thresholding be implemented with a linear filter?

$$g(m, n) = \begin{cases} 255, & f(m, n) > A \\ 0 & \textit{otherwise} \end{cases}$$

Linear filters

- Can thresholding be implemented with a linear filter?

$$g(m, n) = \begin{cases} 255, & f(m, n) > A \\ 0 & \textit{otherwise} \end{cases}$$

Thresholding is a non-linear (“if-else”) operation!

$$L(af + bg) \neq aL(f) + bL(g)$$

Thresholding

Scalars

Images

Linear filters

- Can thresholding be implemented with a linear filter?

$$g(m, n) = \begin{cases} 255, & f(m, n) > A \\ 0 & \text{otherwise} \end{cases}$$

$L(af)$ vs $aL(f)$

$a = 2.0$
Threshold = 120

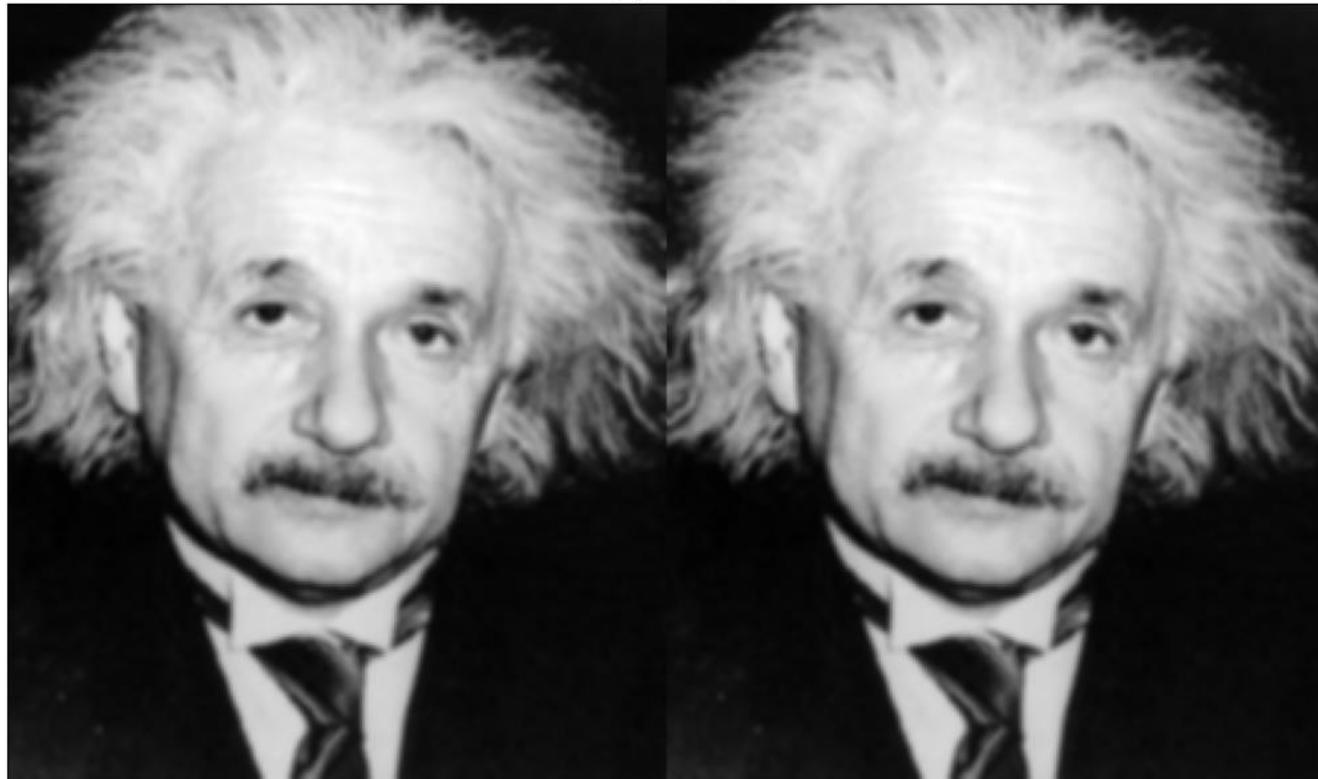


Linear filters

- Can thresholding be implemented with a linear filter?

$$g(m, n) = \begin{cases} 255, & f(m, n) > A \\ 0 & \text{otherwise} \end{cases}$$

L(af) vs aL(f)



**Convolution it's
indeed a linear
operation!!!**

$$L(af) = aL(f)$$

Linear filters

- Can thresholding be implemented with a linear filter?

$$g(m, n) = \begin{cases} 255, & f(m, n) > A \\ 0 & \textit{otherwise} \end{cases}$$

Thresholding is a non-linear (“if-else”) operation!

- There’s no proportional change in the output wrt the input:
 - In a linear filter, if you change a pixel's intensity slightly, the output changes proportionally.
 - However, here, small changes in the pixel intensity near the threshold can result in large, discontinuous jumps in the output.
- The output is not a weighted sum of pixel intensities

Linear filters

- Which one of the following 3x3 filters returns a **positive value** if the **average value of the 4-adjacent neighbors is less than the center**, and a negative value (or zero) otherwise?

(a)

0	$\frac{1}{4}$	0
$\frac{1}{4}$	-1	$\frac{1}{4}$
0	$\frac{1}{4}$	0

(b)

-1	-2	-1
0	0	0
1	2	2

(c)

0	-1	0
-1	4	-1
0	-1	0

Linear filters

- Which one of the following 3x3 filters returns a **positive value** if the **average value of the 4-adjacent neighbors is less than the center**, and a negative value (or zero) otherwise?

Possible Input Images

1	1	1
1	1	1
1	1	1

≤ 0 0 ✓
 Avg of 4-adjacent neighbors is = to the center

1	1	1
1	0	1
1	1	1

≤ 0 1 ✗
 Avg of 4-adjacent neighbors is > than the center

0	0	0
0	1	0
0	0	0

> 0 -1 ✗
 Avg of 4-adjacent neighbors is < than the center

(a)

0	$\frac{1}{4}$	0
$\frac{1}{4}$	-1	$\frac{1}{4}$
0	$\frac{1}{4}$	0

(b)

-1	-2	-1
0	0	0
1	2	2

1 ✗
 1 ✗
 0 ✗

(c)

0	-1	0
-1	4	-1
0	-1	0

0 ✓
 -4 ✓
 4 ✓

If we'd have this kernel with the signs flipped would be ok!

DEMOSTRACIÓN UN POCO MÁS SISTEMÁTICA

Imaginemos que tenemos un patch de entrada genérico (en donde $\{a, b, c, d, e, f, g, h, i\}$ son los valores de intensidad de los píxeles de la imagen):

a b c
d e f
g h i

Ahora trabajemos con el kernel que sabemos que es correcto: el (c).

1) Imaginemos que, tal y como nos indica el enunciado, partimos de que $(b+d+f+h)/4 < e \rightarrow (b+d+f+h) < 4*e \rightarrow$ la respuesta del filtro es >0

Si asumimos eso, y en base al kernel concreto $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$, tendríamos $-b - d - f - h + 4*e = -(b+d+f+h) + 4*e$.

Como sabemos que $(b+d+f+h) < 4*e \rightarrow -(b+d+f+h) < 4*e \rightarrow >0$ Es decir, si $(b+d+f+h)$ sabemos que es menor que $4*e$, entonces $-(b+d+f+h)$ seguirá siendo más pequeña que $4*e$.

2) Ahora imaginemos que, tal y como nos indica el enunciado, $(b+d+f+h)/4 \geq e \rightarrow (b+d+f+h) \geq 4*e \rightarrow$ la respuesta del filtro es ≤ 0

Si asumimos eso, y en base al kernel concreto $\begin{bmatrix} 0 & -1 & 0 \\ -1 & 4 & -1 \\ 0 & -1 & 0 \end{bmatrix}$, tendríamos $-b - d - f - h + 4*e = -(b+d+f+h) + 4*e$.

Si $(b+d+f+h) = 4*e \rightarrow$ Nuestro $-(b+d+f+h) + 4*e$ sería $-(4*e) + 4*e = 0$

Si $(b+d+f+h) > 4*e \rightarrow$ Nuestro $-(b+d+f+h) + 4*e$ sería $-(>4*e) + 4*e < 0$ Es decir, si sabemos que $(b+d+f+h)$ es mayor que $4*e$, sumar ese valor negado a $4*e$ nos tiene que dar un valor negativo.

En otras palabras, ese kernel cumple con todos los requisitos correspondientes a la función buscada.

Probemos ahora con uno de los kernels que sabemos que son incorrectos: el (a), por ejemplo.

1) Según nos indica el enunciado, partimos de que $(b+d+f+h)/4 < e \rightarrow (b+d+f+h) < 4*e \rightarrow >0$ Es decir, así es cómo se debería comportar el filtro buscado.

En base al kernel concreto $\begin{bmatrix} 0 & 1/4 & 0 \\ 1/4 & -1 & 1/4 \\ 0 & 1/4 & 0 \end{bmatrix}$, tendríamos $(b+d+f+h)/4 - e$. Como sabemos que $(b+d+f+h) < 4*e \rightarrow (b+d+f+h)/4 < e$. Si volvemos a $(b+d+f+h)/4 - e$, tendríamos algo "menor que e", al que se le resta "e", y esa operación da negativo. Esto, evidentemente, no es posible (dado que, por definición, el resultado debería ser mayor que cero), por lo que no se cumple la premisa inicial del enunciado.

Let's imagine a 2D Gaussian kernel of size 9×9

- What would the equivalent sigma be (assuming that k has to be the smallest integer greater than or equal to 3σ)?

Let's imagine a 2D Gaussian kernel of size 9×9

- What would the equivalent sigma be (assuming that k has to be the smallest integer greater than or equal to 3σ)?

$$T = 2k + 1 = 9 \rightarrow k = 4$$

$$3\sigma = 4 \rightarrow \sigma = 1.333$$

Let's imagine a 2D Gaussian kernel of size 9×9

- If we wanted to apply this same 9×9 kernel 3 consecutive times, what would the equivalent kernel size be?
 - Note: When you apply the same kernel of size N a total of k times, the equivalent filter size is $N + (k-1) \cdot (N-1)$

Let's imagine a 2D Gaussian kernel of size 9×9

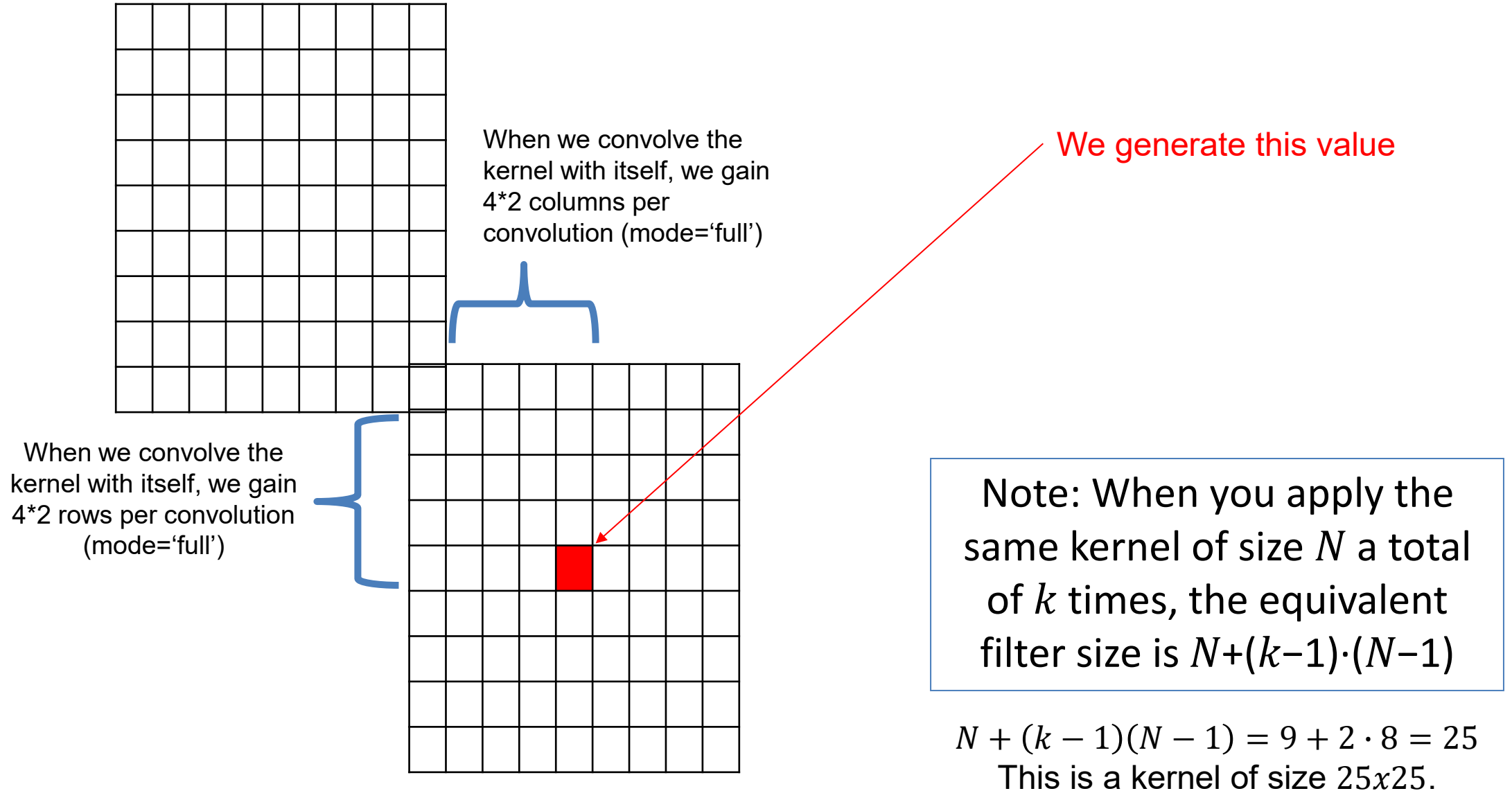
- If we wanted to apply this same 9×9 kernel 3 consecutive times, what would the equivalent kernel size be?
 - Note: When you apply the same kernel of size N a total of k times, the equivalent filter size is $N + (k - 1) \cdot (N - 1)$

The new kernel size would be

$$N + (k - 1)(N - 1) = 9 + 2 \cdot 8 = 25$$

This is a kernel of size 25×25 .

Let's imagine a 2D Gaussian kernel of size 9×9



Let's imagine a 2D Gaussian kernel of size 9×9

- If we want to apply this same 9×9 kernel 3 consecutive times, what would the equivalent kernel size be using the property that the Gaussian is closed under convolution (i.e., convolving N consecutive times with sigma is equal to a single convolution with a Gaussian with $\sigma\sqrt{N}$)? In this case, we look for the filter with the equivalent blurring effect.

Let's imagine a 2D Gaussian kernel of size 9×9

- If we want to apply this same 9×9 kernel 3 consecutive times, what would the equivalent kernel size be using the property that the Gaussian is closed under convolution (i.e., convolving N consecutive times with sigma is equal to a single convolution with a Gaussian with $\sigma\sqrt{N}$)? In this case, we look for the filter with the equivalent blurring effect.

The new kernel size would be

$$\begin{aligned}\sigma_{total} &= \sigma_{ini} \cdot \sqrt{N} = \sigma_{ini} \cdot \sqrt{3} = 1.33 \cdot \sqrt{3} = 2.31 \\ \rightarrow 3 \cdot \sigma_{total} &= 6.93 \rightarrow k = 7 \rightarrow T = 15\end{aligned}$$

$$\begin{aligned}T &= 2k + 1 = 9 \rightarrow k = 4 \\ 3\sigma &= 4 \rightarrow \sigma = 1.333\end{aligned}$$



The kernel with equivalent blurring effect is of size 15×15 .

Is OpenCV using correlation or convolution?

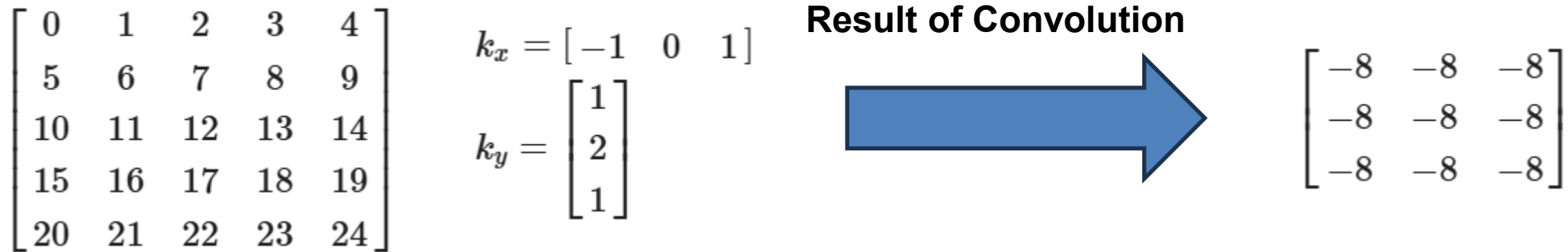
- We know that `cv2.filter2D()` employs correlation.
 - https://docs.opencv.org/4.x/d4/d86/group_imgproc_filter.html#ga27c049795ce870216ddfb366086b5a04
 - “The function does actually compute correlation, not the convolution”

Now, what about `cv2.Sobel()`??

Is OpenCV using correlation or convolution?

- `cv2.Sobel()` also applies correlation.

1. If you do a small test you'll find out:



But... `cv2.Sobel(input, -1, dx=1, dy=0, ksize=3)` yields:

$$\begin{bmatrix} 8 & 8 & 8 \\ 8 & 8 & 8 \\ 8 & 8 & 8 \end{bmatrix}$$


Is OpenCV using correlation or convolution?

- cv2.Sobel() also applies correlation.

2. If you check the documentation (getSobelKernels), this behavior is consistent:

- <https://github.com/opencv/opencv/blob/4.x/modules/imgproc/src/deriv.cpp>

```
if( ksize == 1 )
    kerI[0] = 1;
else if( ksize == 3 )
{
    if( order == 0 )
        kerI[0] = 1, kerI[1] = 2, kerI[2] = 1;
    else if( order == 1 )
        kerI[0] = -1, kerI[1] = 0, kerI[2] = 1;
    else
        kerI[0] = 1, kerI[1] = -2, kerI[2] = 1;
}
```

This is the 1st order derivative kernel of size 3 used by Sobel in OpenCV



Is convolution equivariant to rotation and scaling
(when using isotropic kernels, like Gaussian smoothing)?

First, what does “equivariance” mean?

`convolution(rotation(Img)) = rotation(convolution(Img))`

`convolution(scaling(Img)) = scaling(convolution(Img))`

Is convolution equivariant to rotation and scaling
(when using isotropic kernels, like Gaussian smoothing)?

Original Image



Rotated Image (90°)



1^o Convolution 2^o Rotation



1^o Rotation 2^o Convolution



$$\frac{1}{16} \cdot \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}$$

Yes, it is. But we already knew that.

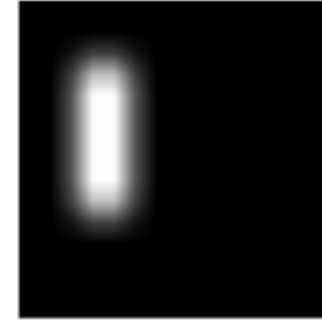


Is convolution equivariant to rotation and scaling (when using isotropic kernels, like Gaussian smoothing)?

Original Image (15, 15)



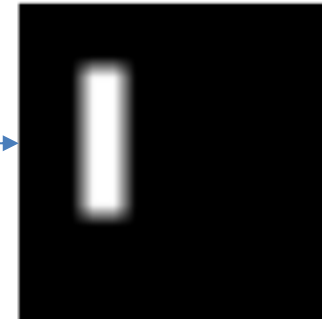
1^o Convolution 2^o Scaling (300, 300)



Scaled Image (300, 300)



1^o Scaling 2^o Convolution (300, 300)



Here, we can't perceive the smoothing effect. The kernel is too small for this image.

No, it is not. Upsampling the smoothed image, amplifies the smoothing effect.

$$\frac{1}{16} \cdot \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}$$

Is convolution equivariant to rotation and scaling
(when using isotropic kernels, like Gaussian smoothing)?

Original Image (15, 15)



1^o Convolution 2^o Scaling (300, 300)



Scaled Image (300, 300)



1^o Scaling 2^o Convolution (300, 300)



$$\frac{1}{16} \cdot \begin{pmatrix} 1 & 2 & 1 \\ 2 & 4 & 2 \\ 1 & 2 & 1 \end{pmatrix}$$

Now we use nearest neighbor interpolation.

Can thresholding be implemented with a linear filter?

- The other day we already concluded that, because it fails superposition/additivity and scaling/homogeneity, **thresholding is a non-linear (“if-else”) operation!**

$$L(af + bg) \neq aL(f) + bL(g)$$

Thresholding

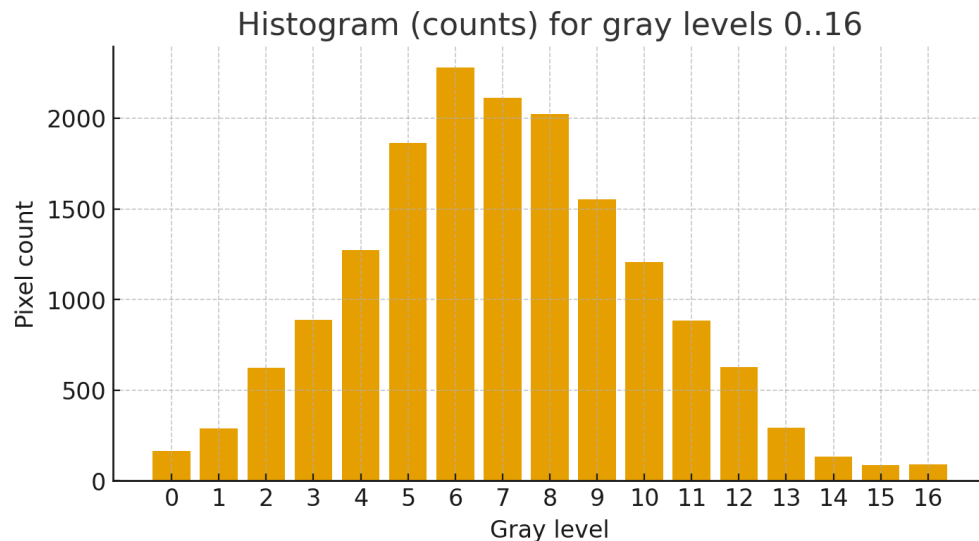
Scalars

Images

$$g(m, n) = \begin{cases} 255, & f(m, n) > A \\ 0 & otherwise \end{cases}$$

Can thresholding be implemented with a linear filter?

- “Since the histogram after thresholding is no longer continuous, the operation is not linear”.
 - Problem:
 - As a function of intensity, **the histogram** (with or without thresholding) is a **piecewise constant function** and has “jumps” at **the bin edges** → **the image histogram is discontinuous**.



$$H(x) = \begin{cases} 163 & \text{for } x \in [0,1) \\ 288 & \text{for } x \in [1,2) \\ 624 & \text{for } x \in [2,3) \\ \dots & \end{cases}$$

Can thresholding be implemented with a linear filter?

- “Since the histogram after thresholding is no longer continuous, the operation is not linear”.
 - Problem:
 - As a function of intensity, **the histogram** (with or without thresholding) **is a piecewise constant function and has “jumps” at the bin edges** → **the image histogram is discontinuous.**
 - Therefore, by itself, **the histogram does not tell us whether the underlying operation is linear or not.**
 - You could visualize the histogram (discontinuous) after convolution (linear operation).

Can thresholding be implemented with a linear filter?

- It is the thresholding operation that is discontinuous:

$$g(m, n) = \begin{cases} 255, & f(m, n) > A \\ 0 & \textit{otherwise} \end{cases} \quad \text{discontinuous at } f(m, n) = A$$

- **In the context of images** (a finite-dimensional space), **if an operator is discontinuous, it cannot be linear.**

Can thresholding be implemented with a linear filter?

- Can a piecewise-defined function be linear?
 - **Yes**, provided that all pieces agree with a **single global linear transformation** on the entire domain (i.e., the piecewise form is merely redundant).

$$\text{Let } f(x) = \begin{cases} a \cdot x & \text{if } x \geq 0 \\ b \cdot x & \text{if } x < 0 \end{cases}$$

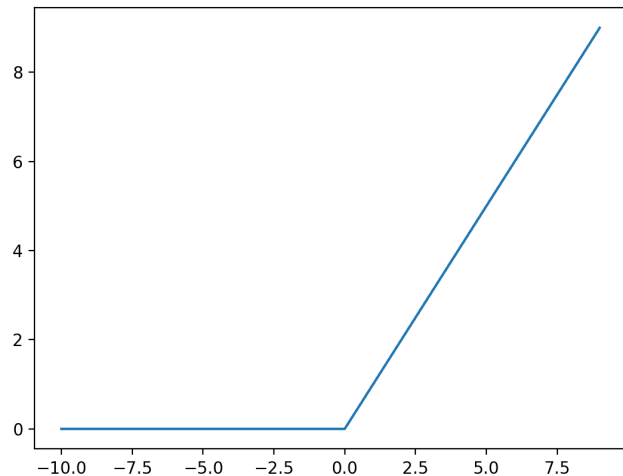
- If f is linear, it must satisfy $f(-x) = -f(x)$ for all x .
- For $x > 0$:

$$\text{Linearity} \Rightarrow f(-x) = -f(x) \Rightarrow -b \cdot x = -(a \cdot x) \Rightarrow a = b$$

Since this must hold for every $x > 0$, we get $a = b$; hence there are not two distinct pieces and $f(x) = a \cdot x$ globally.

Can thresholding be implemented with a linear filter?

- Can a piecewise-defined function be linear?
 - If there's no **single global linear transformation** on the entire domain
→ non-linear function
 - For instance, rectified linear unit (ReLU), which is continuous and non-linear:

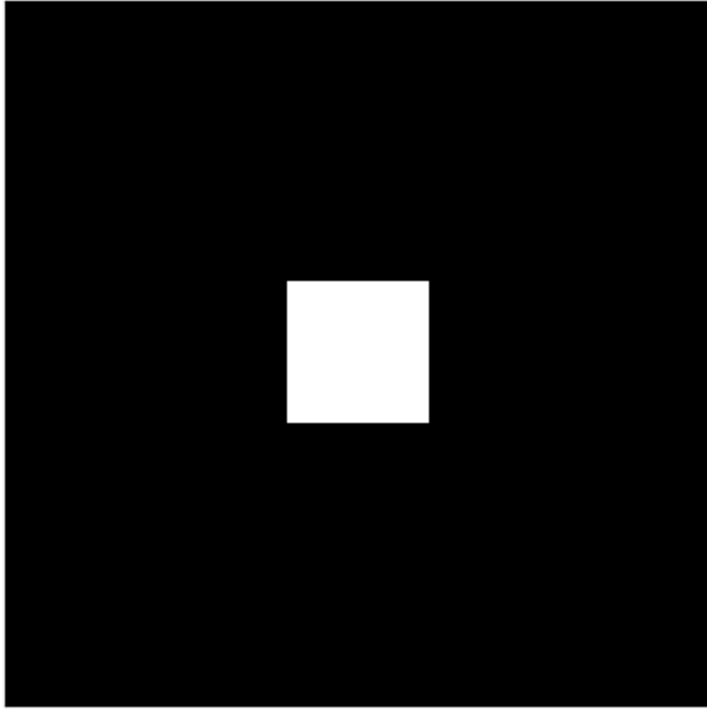


$$f(x) = \begin{cases} x & x \geq 0 \\ 0 & x < 0 \end{cases}$$

There's no "jump". The limit exists and it's 0. It's continuous but non differentiable at $x = 0$.

Fourier Analysis

Original Image

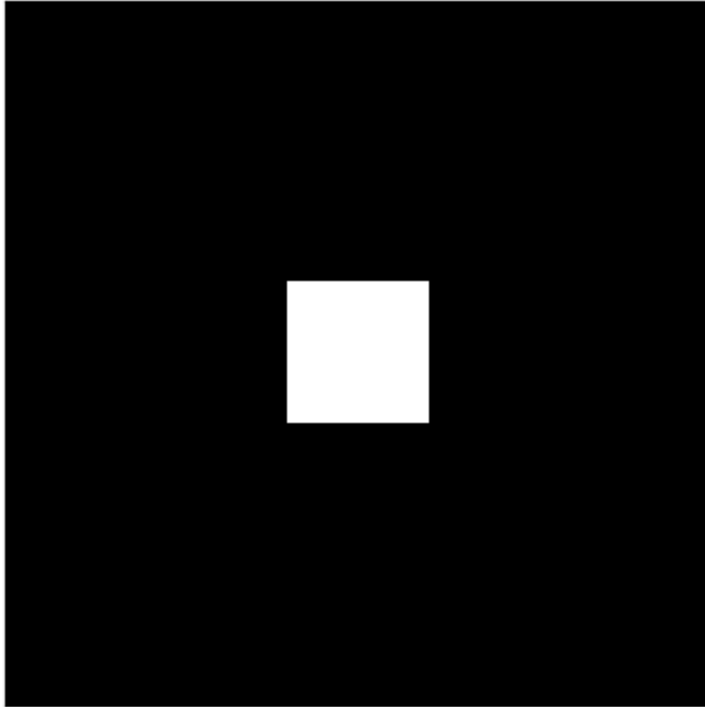


Fourier Transform

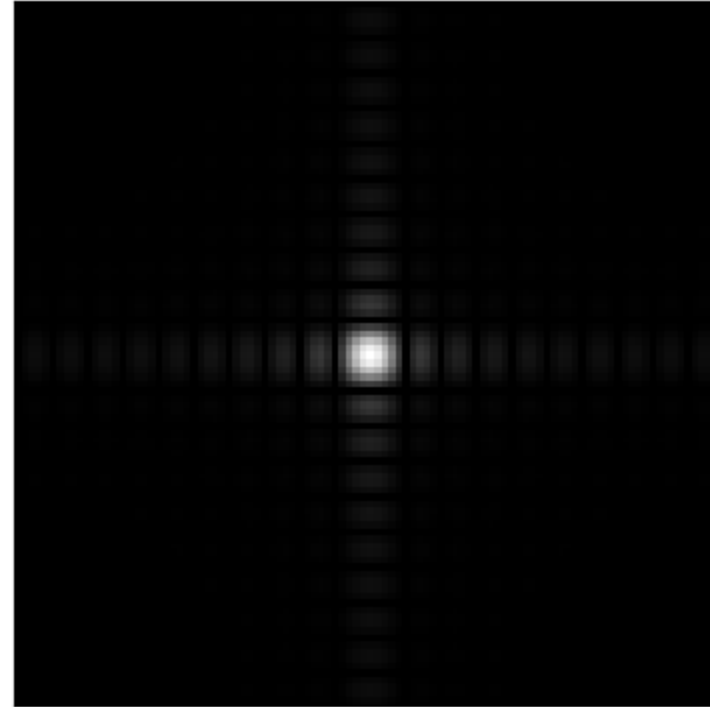


Fourier Analysis

Original Image

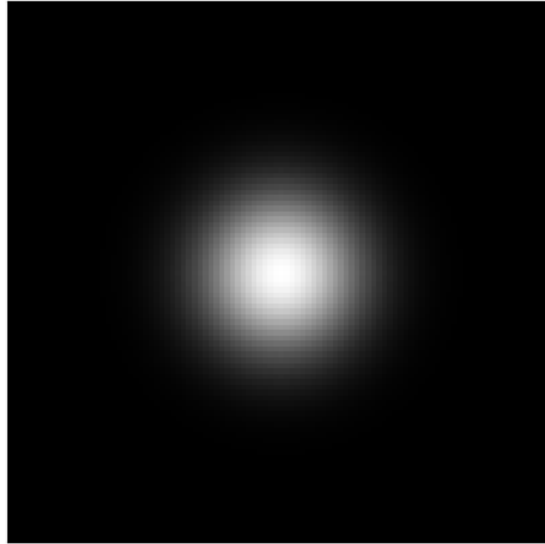


Fourier Transform



Fourier Analysis

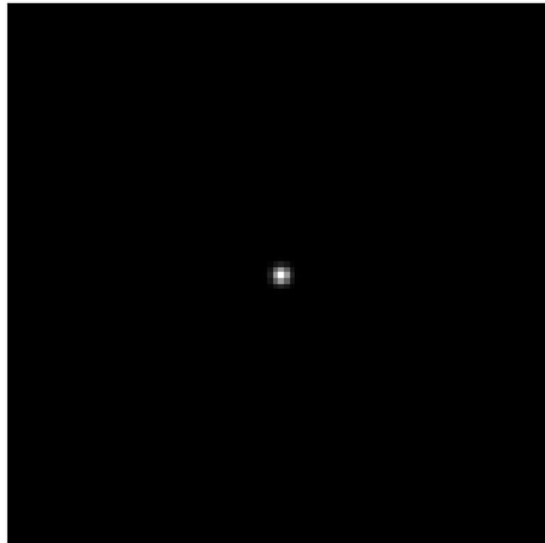
Original Image



Fourier Transform



Original Image

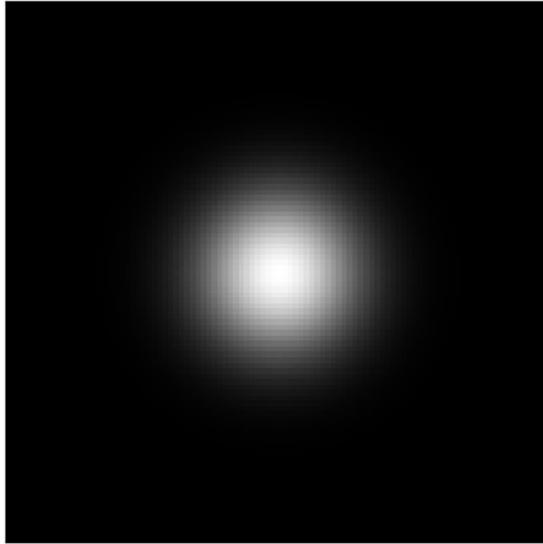


Fourier Transform

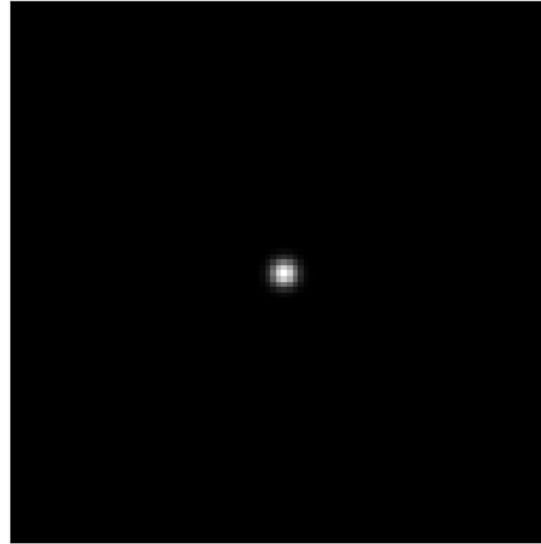


Fourier Analysis

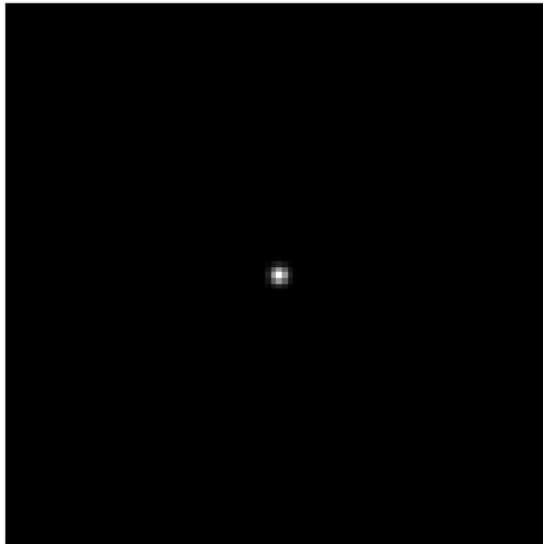
Original Image



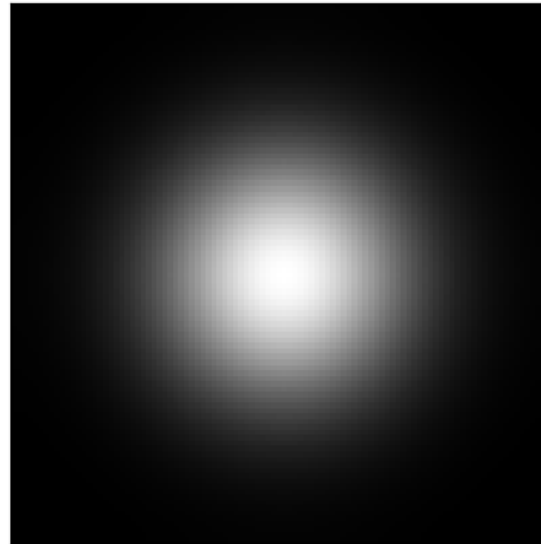
Fourier Transform



Original Image

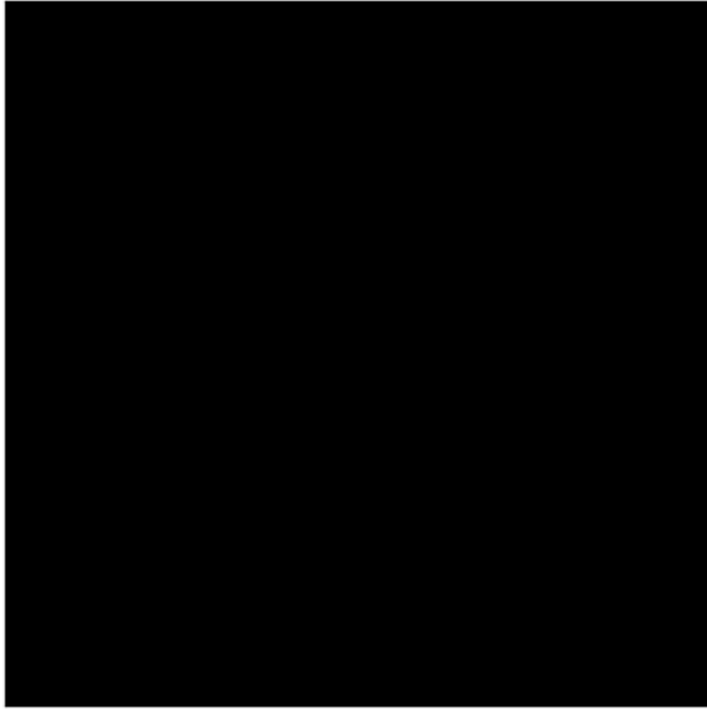


Fourier Transform



Fourier Analysis

Original Image

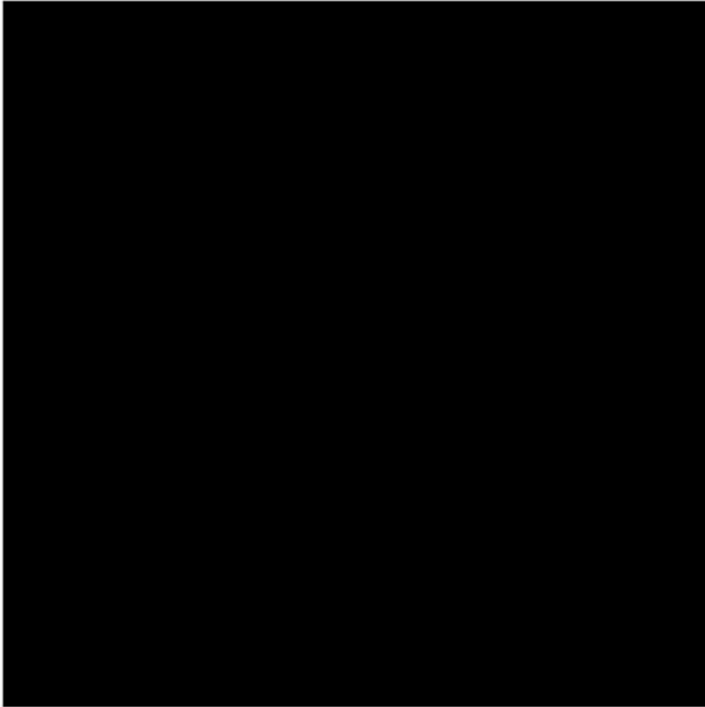


Fourier Transform

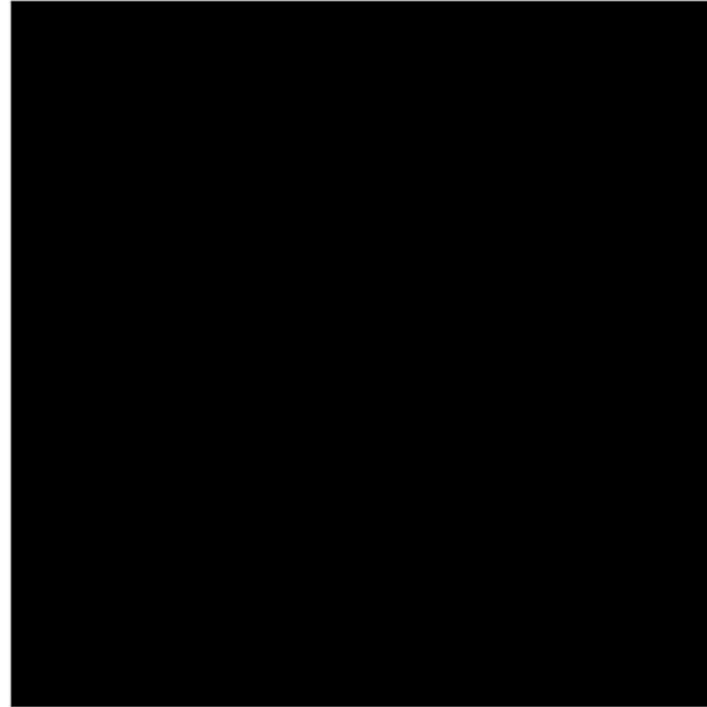


Fourier Analysis

Original Image

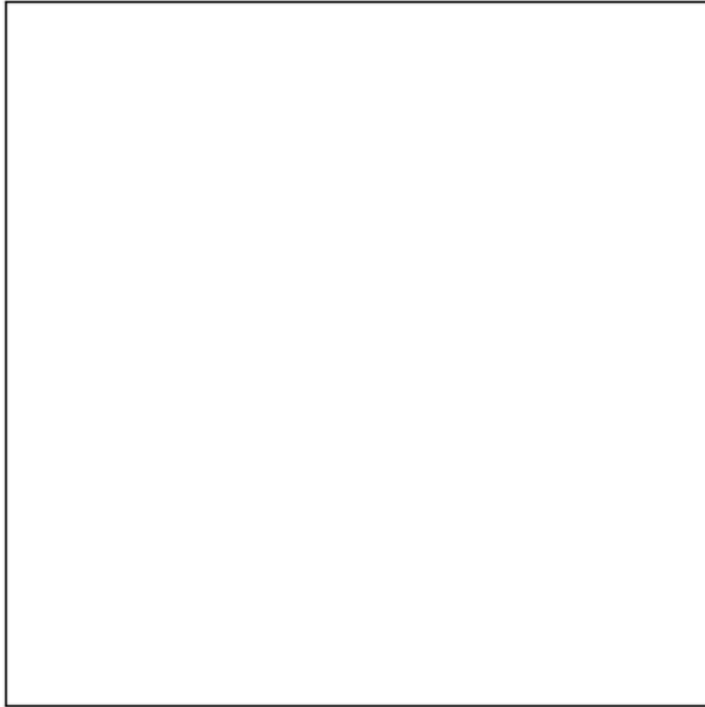


Fourier Transform



Fourier Analysis

Original Image

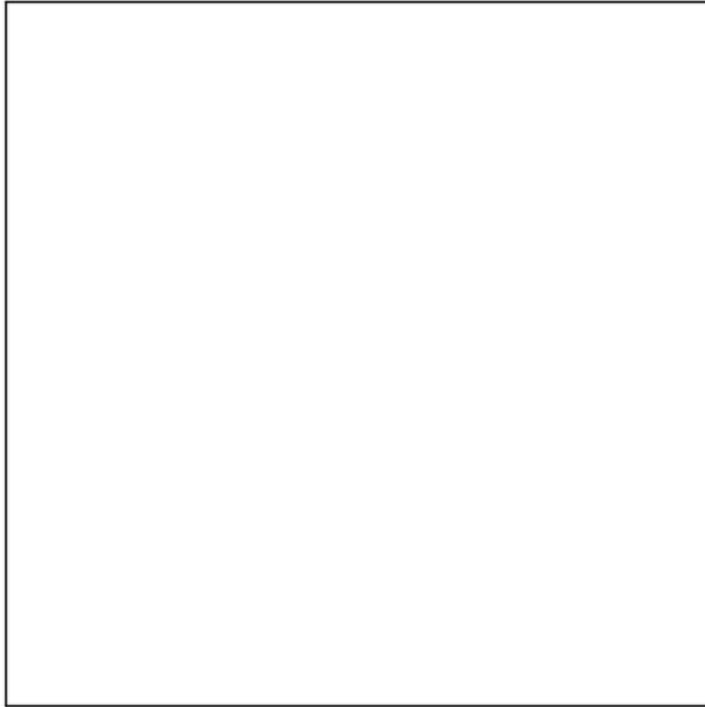


Fourier Transform

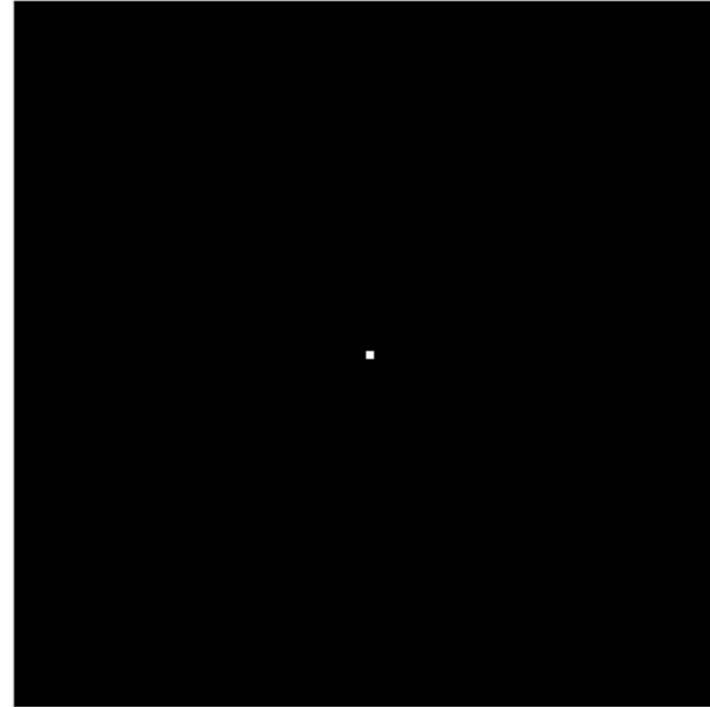


Fourier Analysis

Original Image

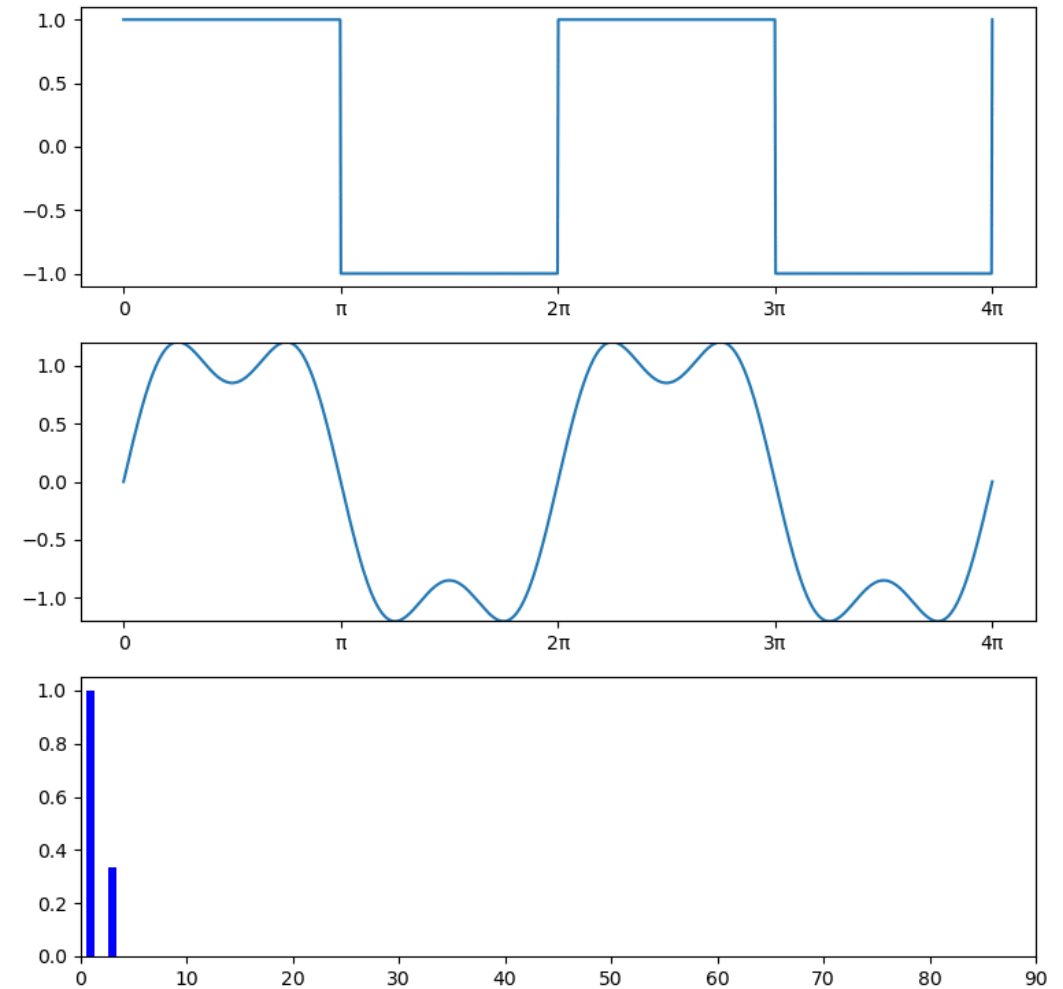
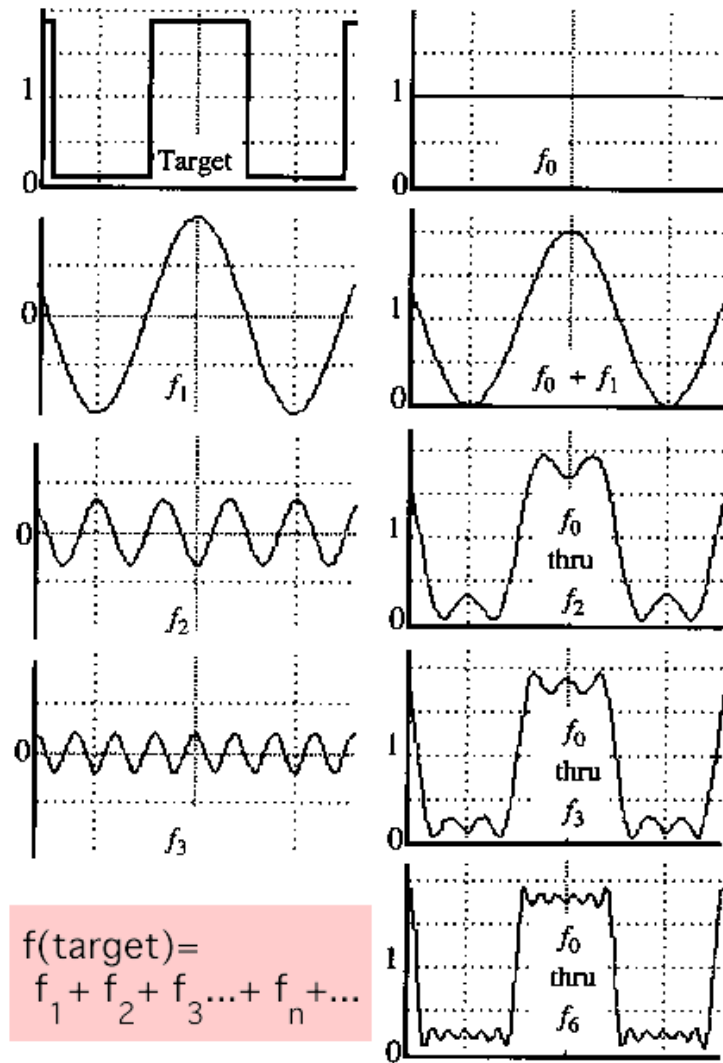


Fourier Transform



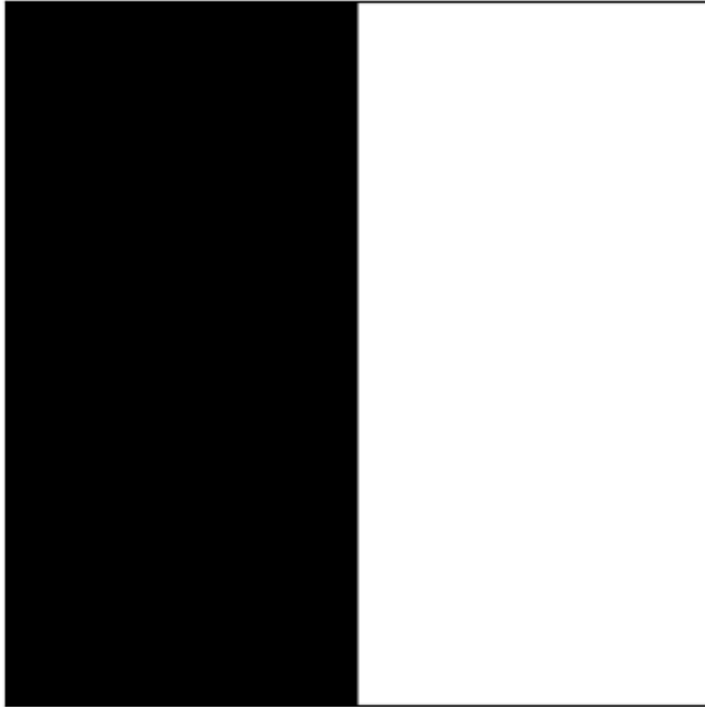
Fourier Analysis

Assuming what we already know:



Fourier Analysis

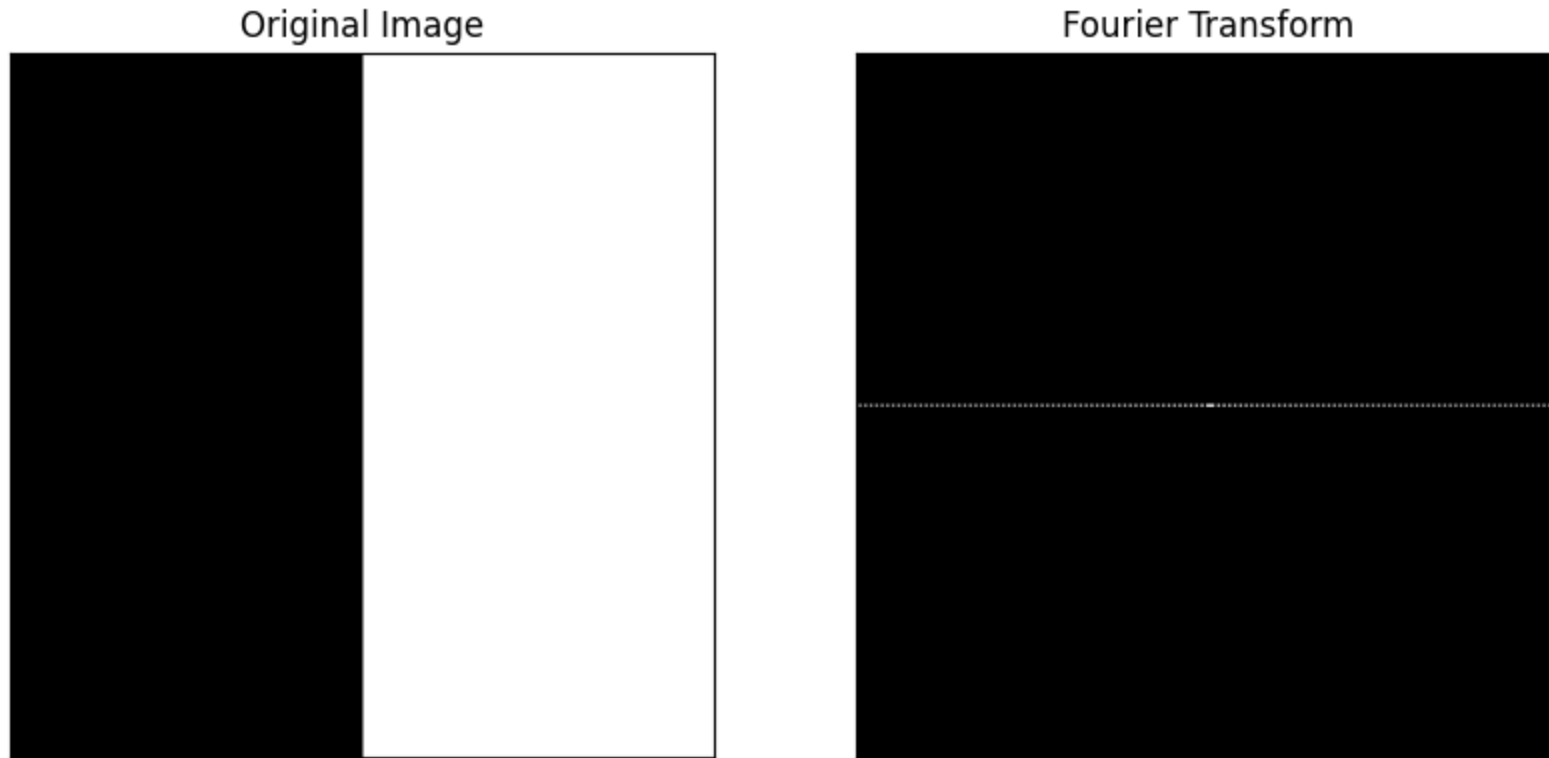
Original Image



Fourier Transform



Fourier Analysis



- **A step is a signal with a discontinuity** (a sudden change in level).
 - Discontinuities can't be produced by a single sinusoid.
 - **You need to add many sinusoids (harmonics) of increasingly higher frequencies.** The more you include, the more abrupt the transition becomes.

What is the exact relationship between large derivatives and high frequencies? Is there a 1:1 correspondence between frequencies (Fourier) and image derivatives?

- Frequency in 2D images: rate of change of gray levels in spatial terms
 - If a change takes many pixels → low frequency
 - If a change happens in few pixels (abrupt) → high frequency
- Given a pixel with a certain derivative value, can we determine exactly how much frequency it contributes to the Fourier image?
 - No.
 - **Derivatives are local – Fourier coefficients are global**
 - Frequency is not a local property of an isolated pixel (it's a collective phenomenon)
 - Each Fourier coefficient (i.e., the value at a point in the frequency image) depends on the values of all pixels in the original image.

What is the exact relationship between large derivatives and high frequencies? Is there a 1:1 correspondence between frequencies (Fourier) and image derivatives?

- But...

based on Parseval's Theorem and the Fourier derivative property, we know that the **total energy of an image's gradient** (the sum of the squared magnitudes of the gradients) is directly proportional to the **energy of the Fourier image weighted by the squared radial frequency**.

$$\sum |\nabla f|^2 \propto \sum \omega^2 |F(\omega)|^2$$

Frequency-domain energy weighted by the squared radial frequency. This emphasizes high frequencies but does not exclude contributions from low ones.

Image interpolation

What would you say is the main effect yielded by the following 2D filter?

$$\begin{bmatrix} 1/4 & 1/2 & 1/4 \\ 1/2 & 1 & 1/2 \\ 1/4 & 1/2 & 1/4 \end{bmatrix}$$



- (a) Gaussian Smoothing
- (b) Edge Detection
- (c) Bilinear Interpolation
- (d) None of the Above

Image interpolation

Why, instead of $\frac{1}{4}$, we don't have a different value, like $\frac{1}{2} \frac{1}{\sqrt{2}} = \sqrt{\frac{1}{8}} \approx 0.35$?

$1/4$	$1/2$	$1/4$
$1/2$	1	$1/2$
$1/4$	$1/2$	$1/4$

Neighbor value

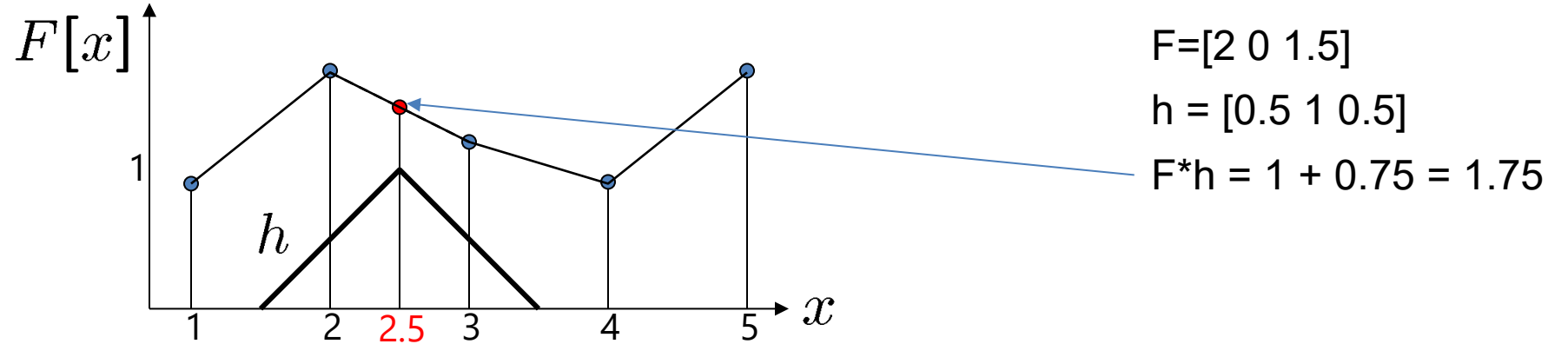
Inverse Euclidean distance
to the center

1) It wouldn't be separable!

$$\begin{bmatrix} 1/2 & 1 & 1/2 \end{bmatrix} \otimes_{outer} \begin{bmatrix} 1/2 & 1 & 1/2 \end{bmatrix} = \begin{bmatrix} 1/4 & 1/2 & 1/4 \\ 1/2 & 1 & 1/2 \\ 1/4 & 1/2 & 1/4 \end{bmatrix}$$

Image interpolation

2) It's not the natural expansion of the 1D interpolation

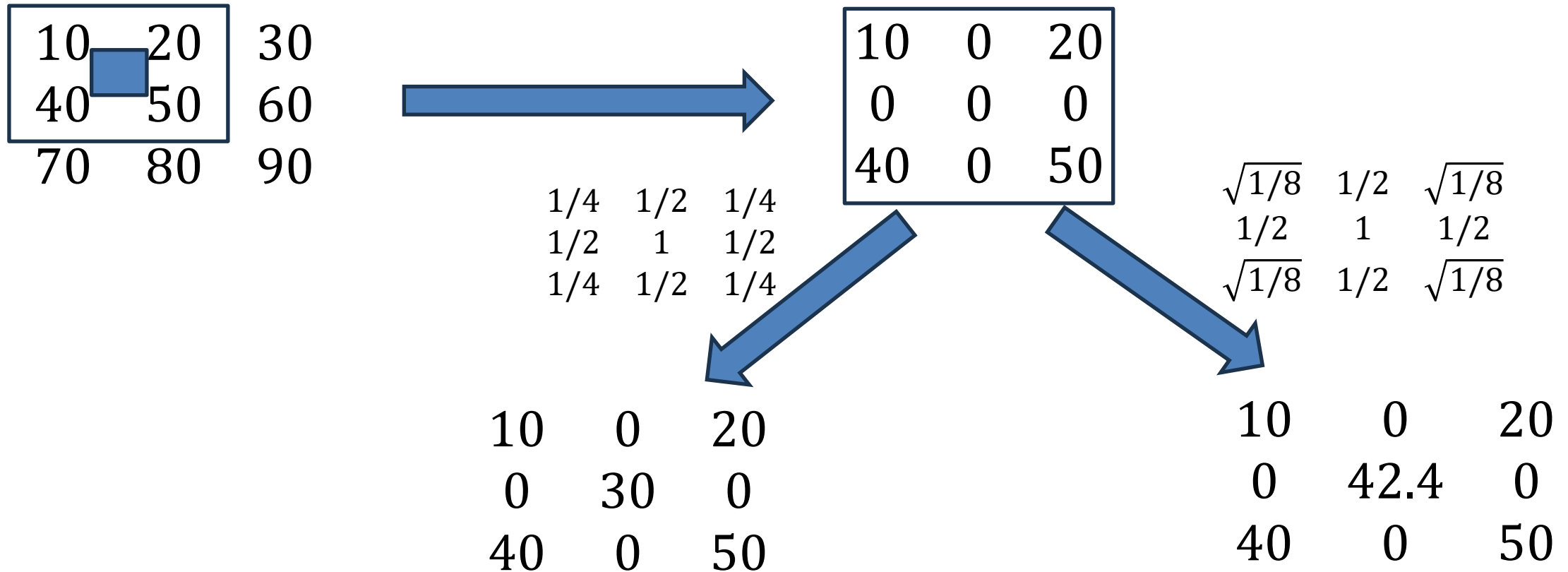


For instance, what if I want to interpolate a value in this position?

10	20	30
40	50	60
70	80	90

Image interpolation

2) It's not the natural expansion of the 1D interpolation



According to your common sense, which one do you think would be better in more scenarios?

